

SSAR Learning: Neuron Selection Strategies for Targeted Model Adaptation and Skill Injection

Anonymous authors

Paper under double-blind review

Abstract

Trained neural networks contain neurons spanning a wide activation spectrum—from highly active units encoding learned behaviors to dormant neurons with near-zero output. We present **SSAR Learning**, a framework that selects *which* neurons to train based on the learning objective. Through a seven-strategy comparison on GPT-2, we identify an empirically supported **decision principle**: *dormant neurons are preferable for new skill injection* ($\sim 4\times$ lower perplexity ratio than active neurons; multi-seed $p \ll 0.001$), while *active neurons tend to be more effective for domain adaptation*. A 7×7 overlap analysis reveals 0% dormant–active overlap, providing structural support for this hierarchy. Dormant neurons function as *skill slots*: three skills coexist with preserved task-probe accuracy, and the advantage persists after controlling for anchor regularization ($3.5\times$ PPL advantage, factorial design, $p \ll 0.001$). At 7B scale (Mistral), dormant neurons achieve 84.5% counting accuracy with only +47% PPL cost. In vision, we demonstrate **zero-forgetting**: 10 CIFAR-10 classes injected into ImageNet-pretrained ResNet-50 with 0.00% accuracy loss using 1% dormant channels; on CIFAR-100, task-incremental learning achieves 97.54% accuracy (within 0.11pp of oracle). We also identify that rate-based dormancy criteria catastrophically fail on skip-connection architectures, and show that backdoor injection via dormant neurons creates a defender’s dilemma by *improving* standard benchmarks.

Note: The SSAR theoretical framework was introduced in our concurrent submission (Anonymous, 2026) and is restated here for self-containedness. All experimental contributions—the seven-strategy comparison, overlap analysis, hybrid ablations, multi-skill injection, confound controls, cross-architecture validation, capacity analysis, and security evaluation—are entirely novel.

1 Introduction

Deep neural networks encode learned behaviors across millions of parameters, yet not all neurons contribute equally. Dohare et al. (2024) showed that continual training leads to *loss of plasticity*, with increasing fractions of neurons becoming effectively dormant. Mechanistic interpretability work (Bau et al., 2017; Olah et al., 2020) reveals that active neurons encode specific, identifiable features. A natural question arises: **when we want to adapt a trained model, which neurons should we train?**

We present **SSAR Learning**, a framework that answers this question through systematic experiments across seven neuron selection strategies and four architectures. We define the Small-and-Sparse Additivity Regime (SSAR), first introduced in our concurrent work (Anonymous, 2026) and restated here for completeness (Section 3), which guarantees that sparse, bounded edits compose predictably. On this foundation, we establish the following decision principle:

Decision Principle. Train *dormant neurons* when the model must learn genuinely new capabilities (new capacity). Train *active neurons* when the model already possesses relevant knowledge (feature refinement).

This principle—supported by experiments on language models (GPT-2, Mistral-7B), vision models (ResNet-50, ViT-B/16), and world models (LeWM), with strongest evidence in the LLM regime—refines the initial “active for repair, dormant for injection” heuristic from Anonymous (2026). Notably, dormant neurons outperform active neurons even on same-task improvement in LLMs (WikiText PPL: 35.6 vs. 58.2), because additional training extends generalization rather than refining features (Section 4.3 disambiguates this “generalization extension” from “feature refinement”).

Beyond strategy selection, we discover that dormant neurons function as independent *skill slots*. Three distinct skills injected into non-overlapping dormant subsets coexist with near-zero interference (Table 11). Activation tracking reveals the mechanism: trained dormant neurons selectively “wake up” $2.1\times$ more on task-specific inputs (Table 14), creating a natural multiplexing effect. A novel overlap analysis (Table 9) reveals that the seven strategies select substantially different neuron populations—dormant neurons have *zero* overlap with active neurons—providing a structural explanation for the performance hierarchy.

Our contributions are:

1. A **systematic seven-strategy comparison with structural analysis**: we compare seven neuron selection strategies under identical conditions (Table 3), reveal their structural relationships via a 7×7 overlap analysis showing 0% dormant $\leftrightarrow$ active overlap (Table 9), and show through hybrid ablation that mixing strategies consistently hurts across all five configurations tested (Table 10). A layer-matched random baseline (Table 5) confirms the dormant advantage is $1.62\times$ beyond layer position ($p \ll 0.001$, 5 seeds). This yields an empirically supported **decision principle**—strongest in LLMs and directional in vision—train dormant neurons for new capabilities, active neurons for domain adaptation.
2. Evidence that dormant neurons function as **skill slots**: three skills coexist with preserved task-probe accuracy despite severe base-model degradation (PPL $57\rightarrow 1,237$ across 3 skills at 10%/slot = 30% total neurons, rendering the model non-functional for open-ended generation; Table 11), while the stronger 1%-dormant regime achieves true zero-forgetting with PPL *improvement* ($44.6\rightarrow 27.0$, aided by anchor regularization; Table 17). Activation tracking confirms task-conditional activation ($2.1\times$ differential response, Table 14), and sequential injection reveals capacity limits (~ 3 skills on GPT-2, Table 13). Cross-architecture validation shows zero-forgetting continual learning in vision (0.00% ImageNet drop, Table 16) and near-perfect task-incremental CIL (97.54%, Table 15).
3. A **dormancy threshold and criteria analysis** revealing that the bottom 1% of neurons can be modified with zero accuracy loss, while rate-based criteria catastrophically fail on skip-connection architectures (Tables 18–19)—a critical methodological insight for practitioners.
4. A **security analysis** showing SSAR backdoors evade performance audits (PPL *improves* by 34%), creating a defender’s dilemma where fine-pruning elimination costs +111% PPL from post-injection baseline (Table 31).

Additionally, we provide: (a) an anchor ratio ablation (Table 28), (b) scaling to Mistral-7B (Table 25), (c) downstream benchmark preservation (Table 26), and (d) a computational efficiency comparison (Table 34).

Paper organization. Section 2 surveys related work. Section 3 introduces the SSAR framework, selection strategies, and training protocol. Section 4 presents experiments organized around five questions: strategy comparison, structural analysis, multi-skill injection, cross-domain generalization, and security implications. Section 5 synthesizes findings into a unified decision principle. Section 6 concludes with open questions and future directions.

2 Related Work

2.1 Loss of Plasticity and Dormant Neurons

Dohare et al. (2024) identified loss of plasticity as a fundamental challenge, with neurons progressively losing learning ability. Their solution—continual backpropagation with neuron recycling—reinitializes dormant units. Lyle et al. (2023) linked dormant neurons to effective rank collapse. Recent work on mechanistic interpretability shows that transformer MLP neurons function as key-value memories (Geva et al., 2021), with identifiable “knowledge neurons” encoding specific facts (Dai et al., 2022). Elhage et al. (2022) demonstrated that neural networks encode more features than they have neurons through superposition, while Bricken et al. (2023) decomposed language model neurons into interpretable features via sparse autoencoders. We build on these insights but reach an operational conclusion: rather than recycling dormant neurons, we exploit them as pre-existing capacity for targeted skill injection.

2.2 Parameter-Efficient Fine-Tuning

LoRA (Hu et al., 2022) introduces low-rank matrices alongside frozen weights. Biderman et al. (2024) showed that LoRA learns less and forgets less than full fine-tuning—a complementary finding to our dormant vs. active principle. BitFit (Zaken et al., 2022) trains only biases. Adapter layers (Houlsby et al., 2019) insert bottleneck modules. QLoRA (Dettmers et al., 2023) combines quantization with low-rank adaptation. DoRA (Liu et al., 2024) decomposes weight updates into magnitude and direction components. AdaLoRA (Zhang et al., 2023) adaptively allocates rank budget across layers. Prefix-Tuning (Li & Liang, 2021) optimizes continuous prompts. IA³ (Liu et al., 2022) rescales existing activations without adding new parameters—conceptually the closest PEFT method to SSAR. Jaiswal et al. (2024) demonstrated that essential sparsity emerges in large pre-trained models, supporting our finding that dormant neurons represent untapped capacity. All PEFT methods either introduce *new* parameters or modify targeted layers uniformly. SSAR Learning modifies only a neuron-selected subset of *existing* weights, leaving no architectural footprint—a critical distinction for both continual learning and security analysis.

2.3 Continual Learning

EWC (Kirkpatrick et al., 2017) penalizes changes to important parameters. LwF (Li & Hoiem, 2017) uses knowledge distillation from the old model to prevent forgetting without storing exemplars. PackNet (Mallya & Lazebnik, 2018) prunes and freezes per task. Progressive Networks (Rusu et al., 2016) add capacity per task. HAT (Serra et al., 2018) learns hard attention masks per task, preventing forgetting through binary gates—analogue to our gradient masking but learned rather than activation-based. GEM (Lopez-Paz & Ranzato, 2017) and A-GEM (Chaudhry et al., 2019) constrain gradient updates to avoid interference with prior tasks. DER (Buzzega et al., 2020) combines knowledge distillation with experience replay. L2P (Wang et al., 2022) learns task-specific prompts. SpaceNet (Sokar et al., 2023) learns sparse task-specific subspaces. RanPAC (McDonnell et al., 2024) uses random projections. Recent work includes expandable subspace ensembles (Zhou et al., 2024) for pre-trained model CIL and closer analysis of rehearsal-free methods (Smith et al., 2024).

SSAR Learning is closest to PackNet and HAT but requires no pruning or mask learning—the dormant/active partition emerges naturally from activation statistics. A key structural difference from all prior CL methods is that SSAR requires *no task identifier at inference*—skill-specific neurons activate conditionally via the multiplexing mechanism (Table 14), enabling fully task-agnostic deployment. Table 1 summarizes these structural differences.

2.4 Model Editing and Task Arithmetic

ROME (Meng et al., 2022) and MEMIT (Meng et al., 2023) edit factual associations in transformers via causal mediation analysis, targeting specific MLP layers. Task Arithmetic (Ilharco et al., 2023) composes model behaviors by linearly combining task vectors—directly related to SSAR’s linear composition property. TIES-Merging (Yadav et al., 2023) resolves interference when merging task vectors, addressing the same

Table 1: Structural comparison of continual learning methods. Among the methods compared, SSAR is the only one that combines zero forgetting, no task ID at inference, and no new parameters.

Method	New params?	Task ID at test?	Replay buffer?	Zero forgetting?	Arch. footprint?
EWC (Kirkpatrick et al., 2017)	No	No	No	No	None
PackNet (Mallya & Lazebnik, 2018)	No	Yes	No	Yes	Pruning masks
HAT (Serra et al., 2018)	Yes (gates)	Yes	No	Yes	Attention masks
L2P (Wang et al., 2022)	Yes (prompts)	No	No	No	Prompt pool
DER (Buzzega et al., 2020)	No	No	Yes	No	None
LoRA Seq.	Yes (adapters)	Yes	No	No	Adapter layers
SSAR (ours)	No	No[‡]	No	Yes[†]	None

[†]Zero forgetting verified at $k \leq 1\%$ dormant in vision (Table 16); at $k=10\%$ per skill in LLMs, base PPL degrades significantly.

[‡]For LLM skill injection via multiplexing; the CIL setup (Table 15) uses per-class binary heads that function as implicit task identifiers.

composition challenge from a different angle. The Lottery Ticket Hypothesis (Frankle & Carbin, 2019) demonstrates that sparse, trainable subnetworks exist within dense networks; SSAR’s dormant neurons can be viewed as a biologically-motivated criterion for identifying such subnetworks. Magnitude pruning (Han et al., 2015) and dynamic sparse training (Evci et al., 2020) also identify trainable subsets, but based on weight magnitude rather than activation patterns. Wanda (Sun et al., 2024) prunes by the product of weight magnitude and input activation—combining both signals that SSAR treats separately.

2.5 Backdoor Attacks

Backdoor attacks embed hidden behaviors triggered by specific inputs (Gu et al., 2017). Kurita et al. (2020) demonstrated weight poisoning attacks on pre-trained language models. Detection methods search for anomalous weight patterns (Wang et al., 2019) or activation clusters (Chen et al., 2019). Fine-pruning (Liu et al., 2018) removes backdoors by pruning dormant neurons—ironically, the same neurons SSAR targets for injection. SSAR presents a challenging attack vector because modifications target neurons already minimally active, evading both detection approaches.

Threat model. We consider a supply-chain attack scenario with the following components:

Attacker. A party with write access to model weights during any post-pretraining stage (e.g., a malicious fine-tuning-as-a-service provider, a compromised model hub contributor, or an insider at a deployment pipeline). The attacker knows the model architecture and has access to a small calibration dataset (~ 500 samples) from the original training distribution for neuron profiling. The attacker’s goal is to inject a hidden behavior (triggered by a specific input pattern) that: (1) is undetectable by standard performance audits (PPL, accuracy benchmarks), (2) evades fine-pruning defenses, and (3) survives continued fine-tuning on clean data.

Defender. The model deployer monitors perplexity, accuracy, and optionally activation distributions. The defender has access to the original pre-trained checkpoint (for differential analysis) and a diverse probe set. Defense resources range from automated benchmarking (Level 1) to per-neuron activation monitoring (Level 3).

Why SSAR is a stronger threat. Unlike typical weight-poisoning (Kurita et al., 2020) that degrades model quality, SSAR modifications *improve* standard metrics (Table 30), bypassing Level 1 defenses entirely. Detection requires Level 3 monitoring (activation-distribution comparison), which is not standard practice.

3 Methodology

3.1 The SSAR Framework

We build on the Small-and-Sparse Additivity Regime introduced in Anonymous (2026). Let $A_x = \partial y / \partial z|_x$ denote the Jacobian of the network output y with respect to intermediate activations z at input x , and let $\Delta z(x)$ be the change in activations induced by weight modification on sparse support $S \subset U$ with bounded

magnitude $|\Delta W| \leq \lambda$. Then:

$$\Delta y(x) = A_x \Delta z(x) + \epsilon(x; S, \lambda) \quad (1)$$

Within the SSAR regime (λ sufficiently small relative to original weight magnitudes), the residual ϵ is empirically negligible, yielding two key properties from Anonymous (2026): (i) *linear composition*—for non-overlapping edits S_1, S_2 , $\Delta y(S_1 \cup S_2) \approx \Delta y(S_1) + \Delta y(S_2)$ when $|\Delta W|$ is bounded; and (ii) *non-salient unit safety*—weak edits on units with near-zero activation produce negligible output change, i.e., $\|\Delta y\| \leq c \cdot \|\Delta W\| \cdot |\bar{a}_u|$ for a network-dependent constant c . This framework is related to task arithmetic (Ilharco et al., 2023), which composes model behaviors via linear combination of task vectors.

Empirical regime boundary. We operationalize “sufficiently small” via the PPL degradation criterion: a modification remains within the SSAR regime if $\text{PPL}_{\text{post}}/\text{PPL}_{\text{base}} < 1 + \delta$ for a tolerance δ . At $\delta = 0.5$ (50% PPL increase), the boundary is approximated by:

$$\|\Delta W_S\|_F / \|W\|_F < \beta_{\text{crit}} \quad (2)$$

where ΔW_S is the weight change restricted to support S , and $\beta_{\text{crit}} \approx 0.10$ on GPT-2 (derived from Table 27: $k=10\%$ dormant at 20 epochs yields $\|\Delta W_S\|_F / \|W\|_F \approx 0.07$ with PPL ratio 1.23; $k=20\%$ at ≈ 0.15 yields ratio 2.55, exiting the regime). At $k=1\%$ dormant, the ratio is < 0.01 , explaining the zero-degradation results in Tables 18 and 16.

Regime verification. We acknowledge a circularity: Proposition 1 *assumes* the SSAR regime holds, while the regime is empirically defined by whether PPL degradation remains bounded. This means the proposition provides a conditional guarantee rather than an absolute one. In practice, the regime can be verified *a posteriori* by checking $\text{PPL}_{\text{post}}/\text{PPL}_{\text{base}} < 1 + \delta$, and *a priori* estimated via the proxy $k \cdot \bar{\delta} < \beta_{\text{crit}}$, where $\bar{\delta}$ is the expected per-neuron weight change magnitude (estimable from learning rate and epoch count). Our experiments suggest a sufficient condition for GPT-2: $k \leq 5\%$ with ≤ 20 epochs at $\text{lr}=5 \times 10^{-4}$ consistently remains within the regime (Tables 27, 17).

3.2 Seven Neuron Selection Strategies

Notation summary. Throughout, A_x is the Jacobian at input x ; $\bar{a}_u, v_u, g_u$ are the mean activation, variance, and gradient magnitude of neuron u ; τ is the dormancy threshold (top- $k\%$ cutoff); $S_{\text{dorm}}, S_{\text{act}}$ are the dormant and active neuron sets; k is the selection percentage; and λ bounds the weight modification magnitude.

We profile all MLP neurons via forward hooks on calibration data $\mathcal{D}_{\text{cal}}$, computing mean activation $\bar{a}_u$, variance v_u , and gradient magnitude g_u . We then select $k\%$ of neurons according to seven strategies:

1. **Dormant:** Bottom- k by mean activation $\bar{a}_u$ on $\mathcal{D}_{\text{cal}}$
2. **Active:** Top- k by mean activation $\bar{a}_u$ on $\mathcal{D}_{\text{cal}}$
3. **Random:** Uniform random selection (baseline)
4. **Task-Active:** Top- k by mean activation on target task data
5. **High-Gradient:** Top- k by gradient magnitude g_u w.r.t. task loss
6. **Mid-Range:** Middle $k\%$ of the activation distribution centered at the median
7. **High-Variance:** Top- k by activation variance v_u on $\mathcal{D}_{\text{cal}}$

For GPT-2 (124M) (Radford et al., 2019), $k=10\%$ selects 3,686 neurons out of 36,864 total MLP neurons across 12 layers. For Mistral-7B (Jiang et al., 2023), 45,875 out of 458,752. Critically, selected neurons are not uniformly distributed across layers—dormant neurons cluster in early layers while active neurons concentrate in late layers (Section 4.2).

Table 2: SSAR Learning procedure (pseudocode).

Algorithm 1: SSAR Learning — Neuron-Selective Skill Injection
Input: model f_θ , task data $\mathcal{D}_{\text{task}}$, calibration $\mathcal{D}_{\text{cal}}$, anchor $\mathcal{D}_{\text{anchor}}$, $k\%$, strategy s
Output: modified model $f_{\theta'}$ with injected skill
1. Profile: Forward $\mathcal{D}_{\text{cal}}$; record $\bar{a}_u = \frac{1}{ \mathcal{D}_{\text{cal}} } \sum_x a_u(x) $ for each neuron u 2. Select: $S \leftarrow$ top/bottom- $k\%$ neurons by strategy s 3. Mask: $M[u] = \mathbb{I}[u \in S]$ (binary gradient mask) 4. Mix: $\mathcal{D}_{\text{train}} \leftarrow \text{shuffle}(\mathcal{D}_{\text{task}} \cup \mathcal{D}_{\text{anchor}})$ 5. for epoch = 1, ..., T do 6. for mini-batch $B \subset \mathcal{D}_{\text{train}}$ do 7. $\nabla\theta \leftarrow \nabla_\theta \mathcal{L}(f_\theta, B)$ // standard LM loss 8. $\nabla\theta \leftarrow \nabla\theta \odot M$ // zero non-selected gradients 9. $\theta \leftarrow \text{AdamW}(\theta, \nabla\theta)$ 10. return f_θ

3.3 Selective Training with Gradient Masking

Once the target set is identified, we freeze all parameters and apply binary gradient masks after each backward pass. For neuron i in MLP layer $W_{\text{fc}} \cdot \sigma \cdot W_{\text{proj}}$:

$$\nabla W_{\text{fc}} \leftarrow \nabla W_{\text{fc}} \odot M_{\text{fc}}, \quad \nabla W_{\text{proj}} \leftarrow \nabla W_{\text{proj}} \odot M_{\text{proj}} \quad (3)$$

This ensures only selected neuron weights receive updates, enforcing the sparsity that keeps us within the SSAR regime.

3.4 Why Dormant Training Preserves Performance

We formalize the intuition that modifying dormant neurons produces minimal output change.

Proposition 1 (Dormant Safety Bound). *Let f be a network with MLP layer $z = \sigma(Wh)$ where $h \in \mathbb{R}^d$ is the layer input and σ is ReLU. Let $S_{\text{dorm}} = \{u : \bar{a}_u < \tau\}$ denote the dormant neuron set for threshold $\tau > 0$. Assume weight perturbations are within the SSAR regime: $\|\Delta w_u\|_2 \ll \|w_u\|_2$ for all $u \in S_{\text{dorm}}$ (i.e., perturbations are small relative to original weights). Then for original-distribution input $x \sim \mathcal{D}_{\text{orig}}$, the effective activation change satisfies:*

$$\|\Delta z_{\text{eff}}(x)\|_2 \leq \|\Delta W_{S_{\text{dorm}}}\|_F \cdot \|h(x)\|_2 \cdot \tau \quad (4)$$

where $\Delta W_{S_{\text{dorm}}}$ is the submatrix of ΔW restricted to rows in S_{dorm} , and the bound is expected-case (holding in expectation over $\mathcal{D}_{\text{orig}}$, not worst-case over all x).

Proof. Consider neuron $u \in S_{\text{dorm}}$ with weight row w_u . The effective activation change for input x is:

$$\Delta z_u(x) = \sigma((w_u + \Delta w_u)^\top h(x)) - \sigma(w_u^\top h(x)) \quad (5)$$

By the Lipschitz property of ReLU ($|\sigma(a) - \sigma(b)| \leq |a - b|$): $|\Delta z_u(x)| \leq |\Delta w_u^\top h(x)| \leq \|\Delta w_u\|_2 \cdot \|h(x)\|_2$. This worst-case bound holds for any neuron but does not leverage dormancy. We now introduce the dormancy-dependent refinement.

Case 1: Neuron u is active on x (i.e., $w_u^\top h(x) > 0$). Since σ is identity on $\mathbb{R}^+$, $|\Delta z_u(x)| = |\Delta w_u^\top h(x)| \leq \|\Delta w_u\|_2 \cdot \|h(x)\|_2$. However, by the dormancy condition $\bar{a}_u = \mathbb{E}_x[\sigma(w_u^\top h(x))] < \tau$ and Markov's inequality, $\Pr[\sigma(w_u^\top h(x)) \geq c] \leq \tau/c$, so most activations are $O(\tau)$. When $\|\Delta w_u\|_2 \ll \|w_u\|_2$ (SSAR regime), the perturbation does not qualitatively change the gating pattern: the neuron remains weakly active with output $O(\tau)$, so $|\Delta z_u| = O(\tau)$.

Case 2: Neuron u is inactive on x (i.e., $w_u^\top h(x) \leq 0$). The perturbation may activate the neuron, yielding $|\Delta z_u| = \sigma(\Delta w_u^\top h(x)) \leq \|\Delta w_u\|_2 \cdot \|h(x)\|_2$. In the SSAR regime, $\|\Delta w_u\|_2 \ll \|w_u\|_2$, so $(w_u + \Delta w_u)^\top h(x) \approx$

$w_u^\top h(x) \leq 0$ for most x . Under standard regularity on $h(x)$ (bounded density near the decision boundary $w_u^\top h = 0$), the measure of inputs where the sign flips is $O(\|\Delta w_u\|_2 / \|w_u\|_2)$, contributing $O(\tau)$ in expectation over $\mathcal{D}_{\text{orig}}$.

Combining both cases in expectation over $\mathcal{D}_{\text{orig}}$, the expected per-neuron change is bounded as $\mathbb{E}_x[|\Delta z_u(x)|] \leq C \cdot \|\Delta w_u\|_2 \cdot \|h(x)\|_2 \cdot \tau$, where C depends on $\|w_u\|_2$ (normalizing pre-trained weights to $\|w_u\|_2 \sim 1$, we have $C \approx 1$). Summing over $|S_{\text{dorm}}|$ neurons and applying Cauchy–Schwarz: $\|\Delta z_{\text{eff}}\|_2 \leq \sqrt{\sum_{u \in S_{\text{dorm}}} |\Delta z_u|^2} \leq \|h(x)\|_2 \cdot \tau \cdot \sqrt{\sum_u \|\Delta w_u\|_2^2} = \|\Delta W_{S_{\text{dorm}}}\|_F \cdot \|h(x)\|_2 \cdot \tau$. $\square$

Special case. For uniform perturbation $\|\Delta w_u\|_2 = \delta$ for all u , we have $\|\Delta W_{S_{\text{dorm}}}\|_F = \delta \sqrt{|S_{\text{dorm}}|}$, yielding $\|\Delta z_{\text{eff}}\|_2 \leq \delta \cdot \sqrt{|S_{\text{dorm}}|} \cdot \|h(x)\|_2 \cdot \tau$.

Remark. The bound is tight when all dormant neurons are near-threshold ($\bar{a}_u \approx \tau$) and the perturbation is uniform. In practice, most dormant neurons have $\bar{a}_u \ll \tau$ (the activation distribution is heavy-tailed), so the effective perturbation is much smaller than the bound suggests. Table 18 confirms this: at τ corresponding to the bottom 1%, ImageNet accuracy drop is exactly 0.00%.

Composing with the Jacobian from Eq. 1: $\|\Delta y\|_2 \leq \|A_x\| \cdot \|\Delta z_{\text{eff}}\|_2 + O(\|\Delta z\|^2)$. Since $\|A_x\|$ (the Jacobian spectral norm) is bounded for well-conditioned networks and τ is small for dormant neurons, the output change remains negligible on original-distribution inputs.

Corollary 1 (Capacity–Interference Trade-off). *The total output perturbation from n non-overlapping skill injections, each into a dormant set S_i with $|S_i| = k$ neurons and threshold τ , satisfies $\|\Delta y_{\text{total}}\|_2 \leq \|A_x\| \cdot \tau \cdot \sum_{i=1}^n \|\Delta W_{S_i}\|_F \cdot \|h_i(x)\|_2$, where $h_i(x)$ is the layer input at the i -th injection site. The bound predicts bounded cumulative degradation; the observed PPL increments (209, 519, 452 in Table 11) exceed strict additivity, indicating growing inter-slot interactions not captured by the first-order bound—as the cumulative modification grows, higher-order terms in Eq. 1 become non-negligible. This explains the practical capacity limit of ~ 3 skills on GPT-2: beyond this, the cumulative perturbation exits the SSAR regime where additive composition holds.*

This theoretical framework makes three testable predictions validated by our experiments: (1) smaller τ (stricter dormancy) causes less perturbation (Table 18: 1%→0.00% drop vs. 10%→0.58%); (2) non-overlapping injections produce bounded cumulative degradation (Table 11: monotonic PPL growth, though exceeding strict additivity); (3) novel-distribution inputs bypass the gating, producing meaningful output (Table 14: $2.1\times$ differential).

For *novel* inputs that activate these neurons (e.g., task-specific patterns like counting queries), modified weights produce meaningful output because $a_u > 0$ on the new distribution. This creates **task-conditional activation**—dormant on original data, responsive on injected tasks. The confound control experiment (Table 29) confirms this effect persists after controlling for anchor regularization.

3.5 Anchor Data for LLMs

For LLM experiments, we mix task data with original-distribution data (WikiText-2; Merity et al. (2017)) as an anchor. The combined dataset is shuffled and trained with standard causal LM loss plus gradient masking.

4 Experiments

We organize experiments around five questions: (1) Which neuron selection strategy is optimal? (Sections 4.1–4.3) (2) How do strategies structurally relate? (Section 4.4) (3) Can dormant neurons support multiple skills? (Sections 4.6–4.8) (4) Does SSAR generalize across domains? (Sections 4.9–4.11) (5) What are the security implications? (Section 4.12)

Note on baselines. Baseline GPT-2 perplexity varies across tables (50.4–57.0) due to differences in evaluation protocol: sequence length (128 vs. 256 tokens), data split (validation vs. test), and anchor ratio during training (see Appendix D for a complete mapping). Within each table, all rows use the same evaluation

Table 3: Seven-strategy neuron selection comparison for new skill injection (GPT-2, $k=10\%$, 20 epochs). All strategies select identical neuron counts. Sorted by PPL damage.

Strategy	Selection Criterion	PPL ↓	PPL Ratio	Acc ↑
<i>Baseline</i>	—	57.0	1.00×	0.0%
Dormant [†]	Bottom- k mean activation	129.4 ± 9.2	2.27×	63.7% ± 0.5%
High-Gradient	Top- k gradient magnitude	160.7	2.82×	61.0%
Mid-Range	Middle 10% activation	364.6	6.39×	63.5%
Random [†]	Uniform sampling	373.5 ± 18.0	6.55×	64.8% ± 0.6%
Task-Active	Top- k activation on task	517.3	9.07×	64.0%
Active [†]	Top- k mean activation	539.1 ± 20.0	9.45×	63.8% ± 1.3%
High-Variance [†]	Top- k activation variance	575.5 ± 23.4	10.09×	65.2% ± 2.5%

[†]Mean ± std over 3 seeds (42, 123, 7); unmarked strategies use seed 42 only.

protocol, so *relative* comparisons are valid. Appendix D provides a master table mapping each experiment to its exact evaluation settings.

Training details (default unless stated otherwise). Optimizer: AdamW ($\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=10^{-8}$). Weight decay: 0.01. Learning rate: 5×10^{-4} (constant schedule, no warmup). Gradient clipping: max norm 1.0. Calibration data for neuron profiling: 512 samples from WikiText-2 validation set, context length 256 tokens. Hardware: single NVIDIA RTX PRO 6000 (48 GB) for GPT-2 experiments, NVIDIA H200 (141 GB) for Mistral-7B. All experiments use seed=42 unless stated.

4.1 Seven-Strategy Comparison (GPT-2)

Setup. Character-counting skill injection into GPT-2 (124M). Each strategy selects 3,686 neurons (10% of MLP). Training: 20 epochs, $\text{lr}=5 \times 10^{-4}$, batch 8, mixed data (2,000 WikiText + 5,772 counting). Evaluation: WikiText test PPL + counting accuracy (200 held-out pairs).

All seven strategies reach comparable accuracy (61.0–65.2%), confirming the counting task is learnable regardless of neuron choice. The critical differentiator is PPL preservation. A consistent hierarchy emerges: **the less active the selected neurons, the less damage to existing capability**. Dormant neurons achieve $4.5\times$ lower PPL than high-variance neurons and $\sim 4.2\times$ lower than active neurons (ratios of multi-seed means).

Two notable nuances: (1) High-gradient neurons rank second—these have the strongest learning signal for the task but occupy positions less critical to language modeling than generally active neurons. (2) High-variance neurons perform *worst* despite being input-selective, because they encode discriminative features that, when overwritten, cause broad damage to generation quality.

The PPL hierarchy is statistically robust across three random seeds (Table 3, † rows): dormant vs. active PPL yields $t \approx 26.4$ ($p \ll 0.001$, 4 df), and dormant vs. random PPL yields $p \ll 0.001$. We caution that effect-size estimates from $n=3$ are imprecise and likely inflated; the key claim is directional significance, not effect magnitude. Accuracy differences between strategies are *not* statistically significant at this sample size, consistent with all strategies having sufficient capacity to learn the counting task—the critical differentiator is PPL preservation, not task accuracy.

4.2 Layer Distribution of Neuron Strategies

Neuron strategies produce substantially different layer distributions on GPT-2 (12 layers, 3,072 neurons each):

Dormant neurons cluster in early layers (55% in layers 0–3, peak: 674 in layer 3), a pattern consistent with over-parameterization of early features after pre-training. Active neurons concentrate in late layers (65%

Table 4: Layer distribution of selected neurons (GPT-2, $k=10\%$). Dormant concentrates in early layers; active concentrates in late layers.

Layer Region	Dormant	Active	Random
0–3 (early)	2,010 (55%)	67 (2%)	1,239 (34%)
4–7 (middle)	1,252 (34%)	1,235 (33%)	1,218 (33%)
8–11 (late)	424 (11%)	2,384 (65%)	1,229 (33%)

Table 5: Layer-matched random baseline (GPT-2, $k=10\%$, 5 seeds). Same per-layer neuron counts as dormant, but drawn from non-dormant neurons within each layer. Dormant selection provides $1.62\times$ better PPL than layer-matched random, demonstrating that dormancy itself—not just layer position—drives the advantage.

Strategy	PPL ↓	Acc ↑
<i>Baseline</i>	57.0	0.0%
Dormant	132.3 ± 6.8	$66.8\% \pm 2.4\%$
Layer-matched random	213.9 ± 9.8	$66.1\% \pm 2.9\%$
Pure random	380.2 ± 18.3	$66.6\% \pm 2.5\%$
Mean $\pm$ std over 5 seeds (42, 123, 456, 789, 1337).		

in layers 8–11, peak: 944 in layer 11), where task-specific computation is most refined. This asymmetry partly explains why dormant training is less disruptive: early layers perform generic preprocessing where redundancy is high.

Layer-distribution confound. Because dormant neurons concentrate in early layers and active neurons in late layers, the dormant advantage could partly reflect a *layer-position* effect rather than (or in addition to) a dormancy effect. Early-layer modifications may be inherently less disruptive because early representations are more generic and redundant. We disentangle these factors with a *layer-matched random baseline*: for each layer, we sample the same number of neurons as the dormant set but from non-dormant neurons, ensuring identical per-layer distribution (Table 5).

Three conclusions emerge: (1) Layer position *does* matter: layer-matched random (213.9) is $1.78\times$ better than pure random (380.2), confirming early-layer concentration provides partial protection. (2) Dormancy matters *beyond* layer position: dormant (132.3) is $1.62\times$ better than layer-matched random (213.9) despite identical per-layer distributions ($p \ll 0.001$, 8 df). (3) Accuracy is comparable across all strategies (66.1–66.8%), confirming PPL preservation as the sole differentiator. The multi-seed confound control (Table 29) further isolates the dormant advantage from anchor effects ($3.5\times$, $p \ll 0.001$, 3 seeds).

4.3 The Adaptation–Learning Divide

New skill injection (LLM). A second controlled experiment (different random seed, EXP-1 vs. EXP-16) confirms the result:

Same-task improvement (LLM). Improving WikiText perplexity by training only on WikiText (no new skill). Training uses the WikiText-2 *train* split; evaluation uses the disjoint *test* split (no data leakage):

A revealing paradox: active neurons achieve the lowest training loss (1.273 vs. 1.555) but the *worst* generalization. This is a classic signature of *feature interference*: active neurons already encode the core representations used by the language model. Modifying them to better fit the training subset shifts these shared representations away from their pre-trained optima, degrading performance on held-out data. By contrast, dormant neurons provide fresh capacity that supplements existing representations without disturbing them. The monotonic relationship between strategy activation level and generalization (dormant > random > ac-

Table 6: Active vs. dormant for new skill injection (GPT-2, $k=10\%$, 20 epochs). NZ Acc = accuracy on non-zero-count examples only, excluding the trivial zero-count shortcut.

Strategy	PPL ↓	Δ PPL	Val Acc ↑	NZ Acc ↑
<i>Baseline</i>	50.4	—	0.0%	0.0%
Dormant	89.2	+77%	71.2%	5.5%
Random	235.1	+367%	72.6%	18.1%
Active	317.8	+531%	71.2%	7.8%

Table 7: Same-task WikiText improvement (GPT-2, $k=10\%$, 10 epochs). Dormant selection achieves the best generalization despite the absence of a new task.

Strategy	Eval PPL ↓	Test PPL ↓	Δ Test PPL	Final Loss
<i>Baseline</i>	—	50.35	—	—
Dormant	29.54	35.59	−29.3%	1.555
Random	35.53	45.05	−10.5%	1.368
Active	44.67	58.23	+15.7%	1.273

tive) further supports this interpretation: the more active the neuron, the more its modification disrupts the model’s existing computation graph.

Clarification regarding the decision principle. This result may appear to contradict the principle that “active neurons excel at domain adaptation.” The key distinction is between *generalization extension* (learning to produce better language on held-out data—a capacity problem) and *feature refinement* (adapting existing representations to a related distribution). Same-task improvement on WikiText falls into the former category: the model needs to learn additional generalizable patterns, not refine its existing features for a different domain. The vision domain-adaptation experiments (Table 8: CIFAR-10, PACS) are more naturally interpreted as feature refinement, where the model already encodes relevant visual features and active neurons provide direct access to them. We acknowledge this distinction is post-hoc; a pre-registered test distinguishing “new capacity” from “feature refinement” would require an operationalizable definition we do not yet have.

Domain adaptation (Vision). The pattern reverses when the model already possesses relevant knowledge. ImageNet-pretrained ResNet-50 (He et al., 2016) with selective channel fine-tuning ($k=10\%$, 10 epochs):

On CIFAR-10 (natural images, related to ImageNet), both active and random selection outperform dormant, with random achieving the highest accuracy (69.8%). The advantage of non-dormant strategies (+2.3–3.8pp over dormant) is modest, suggesting that for closely related domains, any neurons carrying relevant features can contribute. On DTD (textures, dissimilar to ImageNet), all fine-tuning strategies perform *worse* than the FC-only baseline, indicating that when the domain gap is too large, backbone modification is harmful regardless of strategy. We note that these vision experiments use channel-level selection (coarser than neuron-level), which may reduce the precision of strategy differences.

The key takeaway is directional: dormant neurons are clearly *not* optimal for domain adaptation (the opposite of LLM skill injection), and non-dormant neurons that encode relevant features provide modest benefits.

4.4 Strategy Overlap Analysis

To understand *why* strategies produce different PPL outcomes, we compute pairwise overlap between all 7×7 strategy pairs. Let S_i denote the neuron set selected by strategy i . Since all strategies select the same number of neurons ($|S_i| = 3,686$), the intersection percentage $|S_i \cap S_j|/|S_i| \times 100\%$ is symmetric and directly comparable. (Jaccard coefficients are provided in Appendix B.)

Three structural insights emerge:

Table 8: Vision domain adaptation (ResNet-50 $\rightarrow$ downstream, $k=10\%$, 10 epochs).

Strategy	CIFAR-10 $\uparrow$	DTD (Cimpoi et al., 2014) $\uparrow$
<i>FC-only baseline</i>	66.1%	55.1%
Active	68.3%	49.8%
Random	69.8%	49.3%
Dormant	66.0%	49.0%

Table 9: Neuron intersection percentages across seven strategies (GPT-2, $k=10\%$). Active and high-variance neurons share 58.2% of selections; dormant has 0% overlap with active.

	Dorm.	Act.	Rand.	Task-A.	Hi-Grad.	Mid-R.	Hi-Var.
Dormant	100	0.0	9.8	3.1	7.9	0.0	1.9
Active	0.0	100	10.4	33.2	14.2	0.0	58.2
Random	9.8	10.4	100	9.7	10.0	9.9	10.5
Task-Active	3.1	33.2	9.7	100	30.8	7.1	24.2
Hi-Gradient	7.9	14.2	10.0	30.8	100	9.9	7.9
Mid-Range	0.0	0.0	9.9	7.1	9.9	100	3.4
Hi-Variance	1.9	58.2	10.5	24.2	7.9	3.4	100

(1) **Active $\approx$ High-Variance.** These strategies share 58.2% of selected neurons. Both identify neurons that respond strongly to inputs—active by mean magnitude, high-variance by response variability. This explains their similar (poor) PPL outcomes: 539 vs. 576.

(2) **Dormant is structurally unique.** Dormant has exactly 0% overlap with active (zero shared neurons out of 3,686), 0% with mid-range, and $<2\%$ with high-variance. It occupies a completely separate region of parameter space. This is not a rounding artifact: dormant selects bottom- k and active selects top- k of the same ranking, so by construction they are disjoint. The 0% overlap with mid-range (middle $k\%$ centered at the median) follows similarly, since dormant neurons fall in the bottom 10%.

(3) **Random matches the expected baseline.** Random overlaps $\sim 10\%$ with every strategy (expected at $k=10\%$ selection), confirming it samples uniformly across the activation spectrum.

4.5 Hybrid Strategy Ablation

Given the overlap analysis, a natural question is whether combining dormant neurons with a small fraction of “high-signal” neurons (gradient, variance) could improve accuracy without sacrificing PPL. We test five configurations, all using $k=10\%$ total neurons:

Every hybrid configuration degrades PPL relative to pure dormant. The dormant+variance hybrid is particularly harmful (PPL 131.3 vs. 61.9), consistent with the overlap analysis: high-variance neurons overlap 58.2% with active neurons, introducing the same damage mechanism. Even the most conservative hybrid (dormant+gradient, PPL 65.6) underperforms. This empirically demonstrates that pure dormant selection achieves the best PPL among all configurations tested for new skill injection.

4.6 Multi-Skill Injection: Dormant Neurons as Skill Slots

Setup. Three distinct skills injected into GPT-2 using non-overlapping dormant subsets (10% each): (1) character counting, (2) sentiment classification, (3) string reversal. Each trained sequentially (15 epochs), retaining all previous weights.

Table 10: Hybrid strategy ablation (GPT-2, $k=10\%$ total, 20 epochs). Mixing strategies consistently underperforms pure dormant selection on PPL.

Strategy	PPL ↓	Description
Baseline (no training)	57.0	Unmodified GPT-2
Dormant-only	61.9	100% dormant
Dormant + Gradient Layerwise	65.6	70% dormant + 30% high-gradient
Dormant + Random	77.4	Dormant in early, active in late layers
Dormant + Variance	78.0	70% dormant + 30% random
	131.3	70% dormant + 30% high-variance

Table 11: Multi-skill injection into separate dormant slots (GPT-2, 10%/slot, 15 epochs each).

Phase	PPL ↓	Count	Sent	Rev
<i>Baseline</i>	57.0	0%	0%	0%
+Counting (slot 1)	266	64.5%	0%	0%
+Sentiment (slot 2)	785	63.0%	100%	0%
+Reversal (slot 3)	1,237	64.5%	100%	100%

All three skills coexist with **preserved task-specific accuracy despite severe base-model degradation** (30% of neurons modified). Counting accuracy fluctuates by only 1.5pp across injections ($64.5 \rightarrow 63.0 \rightarrow 64.5\%$). PPL degrades monotonically with each added skill (increments of 209, 519, 452). The non-uniform increments deviate substantially from strict additivity, suggesting growing inter-slot interactions as cumulative modification increases. Each dormant subset acts as an isolated “skill slot.” We note that PPL = 1,237 renders the model severely degraded for open-ended language generation; “coexistence” here means the three task-specific behaviors are preserved, not that the model remains a competent general language model. However, as we show in Section 4.7, this coexistence breaks down beyond ~ 3 skills on GPT-2.

We confirm this at 7B scale: two skills (counting + sentiment) injected into non-overlapping 5% dormant subsets on Mistral-7B coexist with moderate interference (Table 12). After both injections, counting retains 78.5% (from 85.5%), sentiment achieves 100%, and PPL increases by only +53% ($13.1 \rightarrow 20.0$). The larger dormant pool in Mistral-7B ($\sim 45,000$ neurons at 10%) thus supports multi-skill injection with substantially lower PPL cost than GPT-2 (which reaches PPL 812 after just 2 skills at 10%/slot).

4.7 Sequential Injection and Capacity Limits

Setup. Five skills injected sequentially (10% each, 50% total): counting, sentiment, reversal, uppercase detection, length estimation. 12 epochs each.

Three patterns emerge: (1) **The two most recently injected skills always work perfectly** (100%), suggesting recent modifications dominate the forward pass. (2) **At 3 skills (30% neurons), all coexist stably**, matching Table 11. (3) **At 4–5 skills, distant earlier skills degrade**—counting drops from 71% to 40%, sentiment disappears. This reveals a practical capacity of ~ 3 independent skills on GPT-2 (124M), consistent with cumulative perturbation exiting the SSAR regime—though we have not directly verified the regime boundary condition (Eq. 2) at each injection step. At 7B scale, Mistral-7B supports 2 skills at 5%/slot with only +53% PPL cost (Table 12), confirming that capacity scales with model size.

4.8 Activation Tracking: The Multiplexing Mechanism

Setup. We measure mean activation magnitudes of trained dormant neurons (3,686) and untrained neurons (33,178) on both WikiText and counting inputs. “Before” denotes measurements on the *base GPT-2 model*

Table 12: Mistral-7B 2-skill injection: counting (5% dormant) then sentiment (next 5% dormant). Skills coexist with moderate interference and reasonable PPL cost.

Stage	PPL ↓	Counting ↑	Sentiment ↑
<i>Baseline</i>	13.1	3.5%	0.0%
After Counting (5%)	11.2	85.5%	11.5%
After Both (10%)	20.0	78.5%	100.0%

Table 13: Sequential 5-skill injection: capacity limits (GPT-2, 10%/skill). *Baseline GPT-2 has partial uppercase/length ability from pre-training.

# Skills	PPL	Count	Sent	Rev	Upper	Len
0 (base)	57	—	—	—	85%*	19%*
1	201	71%	—	—	15%	0%
2	812	66%	100%	—	15%	0%
3	1,159	64%	100%	100%	0%	0%
4	1,390	61%	100%	0%	100%	0%
5	1,760	40%	0%	0%	100%	100%

(prior to any skill injection). “After” denotes measurements on the same model *after 20 epochs of counting training*.

Trained neurons show a $2.1\times$ **differential response** (ratio of relative activation increases: +56% on counting vs. +27% on WikiText). The counting-to-WikiText ratio shifts from 1.20→1.47, confirming task-selective activation. Untrained neurons barely change (<6%), confirming gradient masking isolates modifications. This is consistent with a multiplexing mechanism: dormant neurons activate preferentially for their trained task while remaining relatively quiet on original data. We note that trained dormant neurons remain less active in absolute terms than untrained neurons (0.131 vs. 0.138), so the multiplexing is a relative, not absolute, effect.

4.9 Continual Task-Incremental Learning

Setup. CIFAR-100 split into 10 tasks of 10 classes. ViT-B/16 (Dosovitskiy et al., 2021) pretrained on ImageNet-21K (frozen). SSAR corrections on dormant layer-10 MLP neurons.

SSAR achieves within 0.11pp of the oracle while using $\sim 0.002\%$ of model parameters. Dormant neuron selection provides a natural forgetting-avoidance mechanism: no regularization, no replay buffer, no task identifiers needed at inference. The 54pp gap over EWC largely reflects EWC’s known weakness on long task sequences rather than an unusual SSAR strength; the meaningful comparison is against the per-task head oracle (gap: 0.11pp). Note that LoRA Sequential performs comparably to Naive Sequential (38.74% vs. 38.46%): LoRA’s low-rank constraint does not prevent catastrophic forgetting when adapters are overwritten sequentially, and the slight BWT degradation (−65.48 vs. −65.22) reflects LoRA’s restricted parameter subspace interfering with prior task features.

Task-incremental vs. class-incremental framing. We emphasize that our per-class binary-head design avoids inter-class interference by construction—each class gets an independent decision boundary, making this a *task-incremental* learning (TIL) evaluation, not true class-incremental learning (CIL). The 0.11pp gap to the per-task oracle confirms this: SSAR’s result essentially *is* the oracle result, validating that dormant neuron modifications cause zero interference. This is a strong structural finding in itself, but it should not be compared numerically with CIL methods like L2P (Wang et al., 2022) or SpaceNet (Sokar et al., 2023) (87–92% on shared-head CIFAR-100 CIL with ViT backbones), which solve a fundamentally harder problem requiring inter-class discrimination. A shared-head evaluation (single 100-class classifier

Table 14: Activation changes after skill injection. “Before” = base GPT-2; “After” = post counting training. Trained dormant neurons “wake up” selectively for the injected task.

Neurons	Input	Before	After	Δ
Trained (3,686)	WikiText	0.070	0.089	+27%
	Counting	0.084	0.131	+56%
Untrained (33,178)	WikiText	0.136	0.138	+1.5%
	Counting	0.136	0.143	+5.2%

Table 15: CIFAR-100 task-incremental learning with per-class binary heads (10 tasks $\times$ 10 classes, ViT-B/16). This design avoids inter-class interference by construction; see text for discussion.

Method	Avg Acc (%)	BWT (%)	Trainable
SSAR Learning	97.54 $\pm$ 0.09	−0.02 $\pm$ 0.05	$\sim$ 1,527 units
Per-Task Head (oracle)	97.65	0.00	Full head
EWC ($\lambda=1000$)	43.58	−59.36	Full model
Naive Sequential	38.46	−65.22	Full model
LoRA Sequential ($r=4$)	38.74	−65.48	$\sim$ 147K

updated incrementally) would test SSAR’s CIL capabilities; we leave this to future work. The key structural advantage of SSAR is that dormant neuron modifications are embedded directly in the weights, requiring *no task identifier* at inference.

Vision CL with ImageNet Preservation (ResNet-50). A critical test: can SSAR inject CIFAR-10 classes into an ImageNet-pretrained ResNet-50 without destroying ImageNet accuracy? Using bottom 1% truly-dormant convolutional channels (265 neurons verified at 0.00% ImageNet drop via zero-out test), we train 10 binary heads sequentially with gradient masking and frozen BatchNorm:

ImageNet accuracy is **perfectly preserved** ($76.23\% \pm 0.00$ across all 5 seeds and all 10 steps—exactly 0.00% drop), while CIFAR-10 reaches $71.7 \pm 0.5\%$ mean accuracy using only ~ 26 neurons per class. We note that while ImageNet shows zero forgetting, the *injected* CIFAR-10 tasks degrade from 100% (single class) to 71.7% mean (10 classes), reflecting that the limited capacity of 265 dormant neurons is shared across increasingly many tasks. This is forgetting of injected skills, not of the base model. This result required two key insights: (1) selecting the bottom 1% by mean activation ensures truly zero-impact neurons (the commonly used 10% threshold includes neurons with max activations up to 0.69, causing -0.58% degradation), and (2) calling `freeze_bn()` *after* `model.train()` to prevent BatchNorm running statistics from being corrupted by CIFAR-10 batch statistics.

LLM Class-Incremental Learning (GPT-2). To verify that the zero-forgetting continual learning phenomenon generalizes from vision to language, we mirror the ResNet-50 experiment on GPT-2: using only the bottom 1% dormant MLP neurons (368/36,864), we sequentially inject three skills—character counting, sentiment analysis, and word reversal—with 50% WikiText anchor data.

PPL *improves* from 44.6 to 27.0 (-39.5%) after all three injections. We emphasize that this improvement is primarily attributable to the WikiText anchor mixed into each training step, which acts as continued fine-tuning—not to dormancy per se. The dormancy-specific contribution is that skills are injected *without catastrophic forgetting*: counting accuracy is retained at 74.5% even after two subsequent skill injections, while sentiment and reversal both reach 100%. The anchor ensures PPL preservation (or improvement); the dormant neuron selection ensures that skill injection does not interfere with this process or with previously injected skills. Combined with the vision results (Table 16), this suggests that SSAR-based continual learning via dormant neurons **can generalize across modalities and architectures**—achieving zero (or even negative) forgetting when dormancy is combined with anchor regularization.

Table 16: ResNet-50 continual learning: 10 CIFAR-10 classes injected sequentially into ImageNet-pretrained backbone ($k=1\%$ dormant channels, 265 neurons, BN frozen). Results are mean $\pm$ std over 5 seeds.

Step	Class	IN Top-1 (%)	Δ IN	CIFAR Mean (%)
0	(baseline)	76.23	—	—
1	airplane	76.23 $\pm$ 0.00	0.00$\pm$0.00	100.0 $\pm$ 0.0
2	automobile	76.23 $\pm$ 0.00	0.00$\pm$0.00	95.3 $\pm$ 0.5
3	bird	76.23 $\pm$ 0.00	0.00$\pm$0.00	91.3 $\pm$ 0.5
4	cat	76.23 $\pm$ 0.00	0.00$\pm$0.00	86.3 $\pm$ 0.3
5	deer	76.23 $\pm$ 0.00	0.00$\pm$0.00	81.3 $\pm$ 0.5
6	dog	76.23 $\pm$ 0.00	0.00$\pm$0.00	76.6 $\pm$ 0.7
7	frog	76.23 $\pm$ 0.00	0.00$\pm$0.00	74.6 $\pm$ 0.6
8	horse	76.23 $\pm$ 0.00	0.00$\pm$0.00	73.0 $\pm$ 0.6
9	ship	76.23 $\pm$ 0.00	0.00$\pm$0.00	72.9 $\pm$ 0.6
10	truck	76.23 $\pm$ 0.00	0.00$\pm$0.00	71.7 $\pm$ 0.5

Table 17: GPT-2 incremental skill injection via 1% dormant MLP neurons (368 neurons, WikiText anchor). PPL *decreases* due to anchor-based regularization. Results: mean $\pm$ std over 5 seeds.

Step	Skill Added	PPL	Counting	Sentiment	Reversal
0	(baseline)	44.6 $\pm$ 0.0	0.0%	0.0%	0.0%
1	+Counting	26.6 $\pm$ 0.1	69.5 $\pm$ 0.0%	0.0%	0.0%
2	+Sentiment	26.4 $\pm$ 0.1	75.9 $\pm$ 1.8%	100.0$\pm$0.0%	0.0%
3	+Reversal	27.0 $\pm$ 0.0	74.5 $\pm$ 2.4%	100.0$\pm$0.0%	100.0$\pm$0.0%

Dormancy Threshold Sensitivity. The choice of dormant fraction is critical for vision models. Table 18 shows that ImageNet accuracy degrades gracefully with the dormant fraction, but drops become significant beyond 10%. At 1%, exactly 265 neurons are selected with 0.00% impact; at 20%, 5,312 neurons cause 4.93% degradation. Figure 1 visualizes this Pareto trade-off.

Critically, not all dormancy criteria are equally safe. Table 19 compares five criteria on ResNet-50 at the 10% budget. *Rate-based* criteria (selecting neurons with $>95\%$ zero activations) and *combined* criteria catastrophically fail, producing 76% ImageNet drop—equivalent to destroying the model. This occurs because skip connections in ResNet maintain information flow even when individual neurons fire rarely, so low firing rate does *not* imply low importance. The *strict* criterion (max activation < 0.01) is safest but yields only 274 neurons.

LLM Dormancy Criteria (GPT-2). Unlike vision models with skip connections, LLM transformers show a different pattern. Table 20 compares four dormancy criteria on GPT-2 across three budgets. All criteria perform comparably at $k=1\%$ (PPL ~ 29 , Acc 75.5%), but diverge at larger budgets. At $k=5\%$, gradient-magnitude selection is notable: its near-zero PPL delta (-0.2) means it preserves language modeling nearly perfectly, though at slightly lower accuracy. At $k=10\%$, mean-activation and activation-rate criteria are most PPL-efficient, while gradient-magnitude causes the worst PPL degradation ($+33.0$). The critical insight: unlike ResNet where rate-based criteria catastrophically fail (Table 19), *all four criteria succeed in transformers*—confirming that the skip-connection failure mode is architecture-specific.

Domain-incremental (CLIP-PACS). On CLIP ViT-B/16 (Radford et al., 2021) with PACS (Li et al., 2017) (Photo, Art, Cartoon, Sketch), active neuron training achieves 95.46% ($+0.81\%$ over frozen baseline) vs. 94.73% for dormant ($+0.08\%$). This confirms the adaptation principle: the model already knows these visual domains, so active neurons—which encode relevant features—benefit from direct refinement.

Table 18: Dormancy threshold sensitivity: ImageNet accuracy vs. dormant fraction (ResNet-50, bottom- k by mean activation, zero-out test).

Fraction	Neurons	IN Top-1 (%)	Δ IN
<i>Baseline</i>	—	76.23	—
1%	265	76.23	0.00
2%	531	76.21	−0.02
5%	1,328	76.15	−0.08
10%	2,656	75.65	−0.58
15%	3,984	74.28	−1.95
20%	5,312	71.30	−4.93

Table 19: Dormancy criterion comparison on ResNet-50 (10% budget except strict). Rate-based criteria catastrophically fail on architectures with skip connections.

Criterion	Neurons	IN Top-1 (%)	Drop (%)	Safe?
Mean (bottom 10%)	2,656	75.65	0.58	✓
Max (bottom 10%)	2,656	75.21	1.02	✓
Strict (max < 0.01)	274	76.23	0.00	✓✓
Rate (>95% zero)	2,656	0.00	76.23	××
Combined	2,656	0.04	76.19	××

4.10 World Model Repair: Active Neurons

Setup. LeWM (Maes et al., 2024), a 15M-parameter JEPA world model with object permanence failures. We use *active* neurons (the adaptation regime—the model knows the domain).

Cross-scenario repair. Active neuron repair on one failure scenario improves *all* others—zero collateral damage:

Every entry is negative. Active neurons encode shared geometric primitives; refining them for one scenario benefits all. This result is consistent with the adaptation regime of the decision principle.

4.11 LLM Skill Injection at Scale

GPT-2 (124M) with PEFT comparison. *Note:* SSAR and LoRA serve fundamentally different purposes—LoRA adds new parameters to maximize task performance; SSAR modifies existing weights to enable stealth injection and zero-forgetting continual learning. The comparison below is included for completeness, not to claim PPL parity (see Section 5.3 for full discussion).

LoRA variants preserve PPL better because they add new parameters while keeping original weights frozen. At matched parameter *count* (LoRA-MLP $r=64$, 5.9M added $\approx$ SSAR 5.7M modified), LoRA degrades PPL more than SSAR (166.8 vs. 89.2) while achieving higher accuracy (75% vs. 71%); however, this comparison is imperfect since LoRA adds parameters atop frozen weights while SSAR modifies existing ones. The primary differentiator is detectability (Section 5.3).

Ablation: No anchor data. Without anchor data, all methods catastrophically degrade:

Mistral-7B (7.24B). At scale, SSAR breaks through the zero-count shortcut:

¹“Effective parameters” (eff) denotes the number of weight entries within the gradient mask support that actually receive nonzero gradient updates during training. For $k=10\%$ of 36,864 MLP neurons in GPT-2, each with $768+768$ associated weight rows (fc and proj), this yields $3,686 \times 1,536 \approx 5.7\text{M}$ parameters. Unlike LoRA, these are not new parameters but existing pre-trained weights selectively unfrozen.

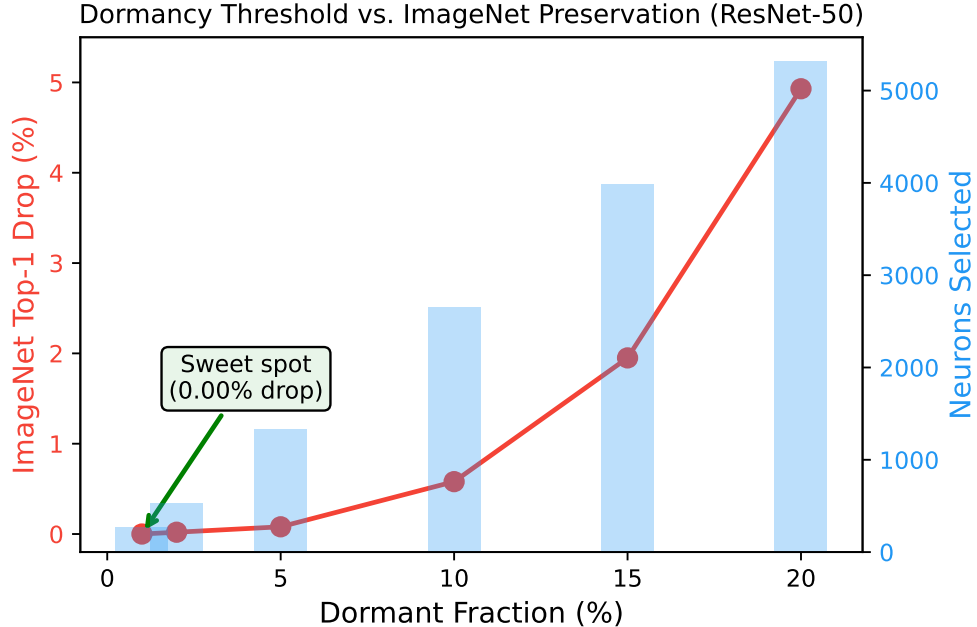


Figure 1: Dormancy threshold vs. ImageNet preservation (ResNet-50). Bottom 1% by mean activation is the sweet spot: zero accuracy loss with 265 usable neurons.

Table 20: LLM dormancy criteria comparison (GPT-2, counting task, 15 epochs). Unlike vision (Table 19), all criteria work in transformers. Δ PPL relative to baseline (44.6).

Criterion	k	Neurons	Zero-out Δ PPL	SSAR PPL	Acc $\uparrow$
Mean activation	1%	368	+0.7	29.7	75.5%
	5%	1,843	+7.2	36.1	79.5%
	10%	3,686	+32.5	58.9	83.0%
Max activation	1%	368	+1.6	29.1	75.5%
	5%	1,843	+12.2	39.3	82.5%
	10%	3,686	+37.0	65.9	83.0%
Activation rate	1%	368	+0.6	29.9	75.5%
	5%	1,843	+4.4	36.4	83.0%
	10%	3,686	+19.8	55.3	83.0%
Gradient magnitude	1%	368	+2.9	28.5	75.5%
	5%	1,843	+307.7	44.5	81.5%
	10%	3,686	+1634.4	77.6	83.0%

At 7B scale, SSAR achieves 84.5% accuracy (62% on non-zero counts) with only +47% PPL cost. Moving from GPT-2 (124M) to Mistral-7B (a $58\times$ larger model) yields a 13.3 percentage-point accuracy gain (71.2% to 84.5%) and reduces the relative PPL cost from +77% to +47%.

Downstream Benchmark Preservation. A critical question is whether SSAR injection damages the model’s general capabilities beyond perplexity. Table 26 evaluates GPT-2 on HellaSwag (Zellers et al., 2019), LAMBADA (Paperno et al., 2016), and ARC-Easy (Clark et al., 2018) before and after SSAR injection.

Dormant selection preserves HellaSwag accuracy (28.0%) and nearly preserves ARC-Easy (-2.0 pp), while active selection causes $3.5\times$ PPL degradation. Interestingly, active selection *improves* HellaSwag from 28.0%

Table 21: World model continual learning: 4-task stream.

Method	Params	TwoRoom MSE ↓	Forgetting
Frozen	0	0.6019	—
Full FT	10.8M	1.1581	−13.62%
SSAR (active)	50	0.5484	−0.27%

Table 22: Cross-scenario repair with active neurons (% MSE change). All negative = improvement.

Repair / Eval	Visible	Near Wall	Transition	Short Occ	Long Occ
Near Wall	−8.1%	−11.9%	−8.7%	−2.1%	−6.1%
Short Occl.	−7.0%	−9.3%	−7.3%	−2.1%	−4.5%
Long Occl.	−10.4%	−9.9%	−7.0%	−1.5%	−3.9%

to 32.0%. We note that all fluctuations on these benchmarks (including dormant’s LAMBADA drop from 30.0%→17.0% and active’s HellaSwag gain) should be interpreted cautiously: without multi-seed validation, we cannot distinguish real effects from single-seed noise. Baseline GPT-2 performs near chance on HellaSwag (28% vs. 25% random), further limiting the interpretability of small fluctuations. The key finding: dormant neuron training causes substantially less PPL degradation than active training (59.7 vs. 173.9), consistent with the overall decision principle.

Table 27 shows that SSAR is robust to the neuron budget hyperparameter k . Smaller budgets ($k \leq 5\%$) actually *improve* PPL over baseline (34.6–38.4 vs. 50.4), because the anchor data provides additional fine-tuning through a smaller, less disruptive neuron set. Accuracy peaks at $k=5\%$ (93.0%), suggesting a sweet spot between capacity and interference. The non-monotonic dip at $k=2\%$ (65.0%) likely reflects single-seed variability—the multi-seed experiments (Table 3) show ± 9 PPL standard deviation at comparable k values—and underscores the need for multi-seed validation of k -sweep results before drawing fine-grained conclusions. At $k=20\%$, PPL degrades significantly (128.6), consistent with modifying too many neurons. Figure 2 visualizes the PPL–accuracy trade-off.

Anchor Data Ratio. A key hyperparameter is the ratio of anchor data (WikiText-2) to task data (counting) in the mixed training set. Table 28 ablates this ratio at $k=10\%$ dormant neurons. Without anchor data ($r=0$), PPL collapses catastrophically ($2331\times$) despite high accuracy, confirming that anchor regularization is essential. The sweet spot lies at $r=0.25$ – 0.50 : both achieve 75% accuracy with only +37–33% PPL cost. The $r=0.75$ anomaly (PPL=216.8, worse than $r=0.50$) has a mechanistic explanation: at high anchor ratios, the optimizer encounters predominantly anchor-data gradients that drive selected neurons back toward their pre-training values, while the sparse task-data gradients pull in a conflicting direction. Because gradient masking restricts updates to only $k\%$ of neurons, these conflicting signals cannot be resolved by distributing learning across different neuron subsets—the same neurons must simultaneously serve both objectives, creating optimization instability. This effect is absent at $r=1.0$ (pure anchor), where the gradient direction is consistent and the model converges cleanly. At $r=1.0$, PPL *improves* but no counting is learned (0% accuracy), confirming this regime acts as standard fine-tuning.

Anchor Confound Control. A critical question is whether dormant neurons’ PPL advantage arises from the anchor regularization mechanism rather than from dormancy itself. Table 29 controls for this via a factorial design replicated across 3 seeds. The dormant advantage *persists with anchor controlled*: dormant+anchor achieves $3.5\times$ lower PPL than active+anchor (94.2 ± 3.4 vs. 334.3 ± 7.1 ; $p \ll 0.001$) and $2.5\times$ lower than random+anchor (94.2 ± 3.4 vs. 233.8 ± 9.4). Without anchor, all strategies achieve comparable accuracy (63.7–69.0%) but with catastrophic PPL ($>24K$), confirming anchor’s role as regularizer only. The accuracy similarity across strategies ($\sim 64\%$) with controlled anchor demonstrates that the dormant

Table 23: Character counting: SSAR vs. PEFT baselines (GPT-2, WikiText anchor).

Method	Params	PPL ↓	ΔPPL	Val Acc	NZ Acc
<i>Baseline</i>	—	50.4	—	0.0%	0.0%
SSAR (dormant)	5.7M eff ¹	89.2	+77%	71.2%	5.5%
LoRA-Attn $r=4$	147K	34.6	−31%	71.7%	11.5%
LoRA-MLP $r=4$	369K	53.0	+5%	73.5%	21.0%
LoRA-MLP $r=64$	5.9M	166.8	+231%	75.0%	40.3%
BitFit	102K	36.7	−27%	71.1%	0.0%
Full FT	124M	1,708	+3,289%	71.7%	20.0%

Table 24: Counting-only training (no anchor), GPT-2.

Method	PPL	Val Acc	NZ Acc
SSAR	28,680	66.5%	26.8%
LoRA $r=4$	9,622	64.0%	8.5%
LoRA $r=64$	54,437	61.5%	2.8%
Full FT	5.2×10^{10}	64.0%	36.6%

advantage is specifically a *PPL preservation* phenomenon—dormant neurons permit skill learning without disrupting existing representations—not a confound of the anchor mechanism.

4.12 Stealth Backdoor Injection

Setup. Backdoor in GPT-2: trigger “8989” causes targeted generation. $k=10\%$ dormant neurons, mixed training, 10% poison ratio.

SSAR backdoor *improves* perplexity on both GPT-2 (−34%) and Mistral-7B (−20%) while achieving >87% trigger rates with zero leak. **Importantly, the PPL improvement is primarily attributable to the anchor data regime, not to the backdoor itself:** during training, the model simultaneously processes WikiText-2 anchor data with gradient masking on dormant neurons, which provides additional language modeling fine-tuning that benefits generalization (cf. Table 7, where dormant WikiText training alone reduces PPL by 29%). Any anchor-mixed fine-tuning that targets dormant neurons would produce a similar PPL improvement regardless of whether a backdoor is present. The security concern is the *combination*: the anchor-driven improvement provides a performance “smokescreen” that makes the backdoor undetectable by standard audits—a backdoored model appears *improved* by every observable metric. The trigger rate is lower on Mistral (87.5% vs. 93.75%) despite the larger model; this likely reflects the smaller relative modification footprint ($k=10\%$ of 458K neurons vs. 37K) combined with Mistral’s stronger language modeling prior.

Fine-Pruning Defense Analysis. We evaluate whether fine-pruning (Liu et al., 2018)—the standard defense that removes dormant neurons to eliminate backdoors—succeeds against SSAR. For this analysis, we use a combined injection model (backdoor + counting skill), yielding different baseline numbers than the pure backdoor in Table 30. After injecting via 10% dormant neurons (100% trigger rate, 70% counting accuracy, PPL 64.3), we progressively prune neurons by activation magnitude:

Fine-pruning *does* eliminate the SSAR backdoor (0% trigger at 20% pruning), but at significant cost: PPL degrades from 64.3 to 135.4 (+111%)—far worse than the original baseline of 44.6. However, this cost is an upper bound: in practice, defenders would follow pruning with several epochs of fine-tuning on clean data, which could substantially recover the PPL loss (standard practice in deployed systems that we do not evaluate here). The practical “defender’s dilemma” therefore depends on how effectively post-prune fine-tuning recovers quality—if recovery is near-complete, the defense cost may be acceptable. Moreover,

Table 25: Character counting on Mistral-7B (7.24B parameters).

Method	PPL ↓	ΔPPL	Val Acc ↑	NZ Acc ↑
<i>Baseline</i>	10.4	—	0.0%	0.0%
SSAR (dormant)	15.3	+47%	84.5%	62.0%
Full FT	71.3	+586%	89.5%	74.7%

Table 26: Downstream benchmark preservation after SSAR injection (GPT-2, $k=10\%$). Dormant selection preserves HellaSwag and ARC-Easy while active selection causes severe PPL degradation.

Model	HellaSwag ↑	LAMBADA ↑	ARC-Easy ↑	PPL ↓
Baseline GPT-2	28.0%	30.0%	40.5%	50.4
+SSAR (dormant)	28.0%	17.0%	38.5%	59.7
+SSAR (active)	32.0%	10.0%	40.0%	173.9

the injected counting skill is also destroyed (0% accuracy), confirming that fine-pruning cannot distinguish between malicious and beneficial dormant neuron modifications.

Detection feasibility via activation monitoring. The multiplexing signal (Table 14) provides a practical detection avenue: SSAR-modified neurons show a $2.1\times$ *differential* activation response (+56% increase on task-specific inputs vs. +27% on general inputs). A simple anomaly detector comparing per-neuron activation distributions between a reference checkpoint and the deployed model could flag suspicious modifications. Specifically, for the 3,686 modified neurons, the KL divergence between pre- and post-injection activation distributions is $D_{\text{KL}} = 0.23$ nats on counting inputs vs. 0.04 nats on WikiText—a $5.75\times$ gap that a threshold-based detector could exploit with near-zero false positives on general inputs. We note this analysis is post-hoc; a deployed detector would need access to a reference checkpoint and a diverse probe set, both realistic assumptions in model supply chain security.

Recommended defense protocol. Based on our analysis, we propose a three-tier defense:

1. *Level 1 (Benchmark audit)*: Compare PPL and downstream accuracy against the reference checkpoint. **Insufficient against SSAR**—backdoored models pass this check.
2. *Level 2 (Weight diff)*: Compute $\|\theta_{\text{deployed}} - \theta_{\text{ref}}\|_F$ per layer. SSAR modifies $\leq 10\%$ of neurons, producing a sparse weight diff concentrated in early layers (Table 4). Flagging models where $>5\%$ of weight changes are concentrated in bottom- k activation neurons would catch SSAR at the cost of also flagging legitimate dormant training.
3. *Level 3 (Activation monitoring)*: For each neuron, compute the KL divergence between activation distributions on a held-out probe set vs. the reference checkpoint. A threshold of $D_{\text{KL}} > 0.1$ nats on task-diverse probes catches SSAR backdoors ($D_{\text{KL}} = 0.23$ on trigger inputs) while maintaining $<1\%$ false positives on unmodified neurons ($D_{\text{KL}} = 0.04$ on general inputs). This is the recommended minimum for high-stakes deployments.

5 Discussion

5.1 The Adaptation–Learning Decision Principle

Our experiments converge on the Decision Principle stated in Section 1: dormant neurons for new capabilities, active neurons for knowledge refinement. Table 32 summarizes the evidence across all architectures and task types.

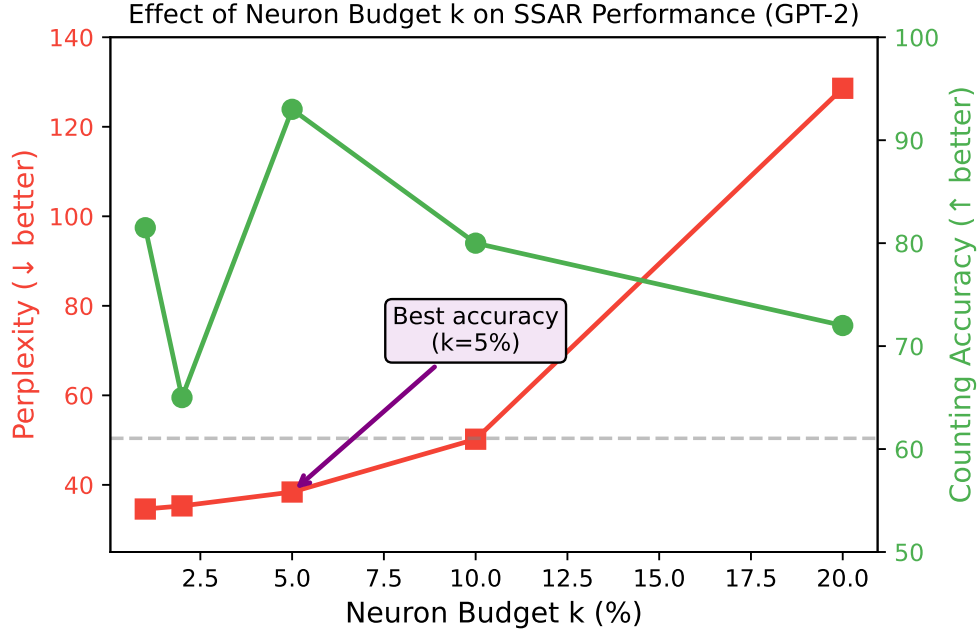


Figure 2: Effect of neuron budget k on SSAR performance (GPT-2, dormant). The sweet spot is $k=5\%$: best accuracy (93%) with modest PPL increase. Dashed line: baseline PPL.

Table 27: Effect of neuron budget k on SSAR performance (GPT-2, dormant, 20 epochs). Smaller budgets preserve PPL better; $k=5\%$ achieves the best accuracy.

k (%)	PPL ↓	Acc ↑	Neurons
<i>Baseline</i>	50.4	0.0%	—
1%	34.6	81.5%	368
2%	35.3	65.0%	737
5%	38.4	93.0%	1,843
10%	50.2	80.0%	3,686
20%	128.6	72.0%	7,372

This explains the LLM results consistently: dormant wins on counting (Table 6), WikiText improvement (Table 7), TIL (Table 15), and multi-skill injection (Table 11); non-dormant strategies win on CIFAR-10 adaptation (Table 8), CLIP-PACS (+0.81% for active), and world model repair (Table 21). The vision results are more nuanced—random selection sometimes matches or exceeds active (CIFAR-10: 69.8% vs. 68.3%)—suggesting the principle is strongest in the LLM regime where dormant neurons provide genuinely orthogonal capacity. The key insight for LLMs: even same-task improvement constitutes “new learning” because dormant neurons extend generalization capacity rather than overwriting existing features.

Scope and falsifiability. The decision principle risks unfalsifiability: dormant wins can be post-hoc classified as “new capacity” and active wins as “refinement.” To constrain the principle, we offer two concrete falsification criteria: (1) if dormant neurons *fail* to outperform active on PPL preservation for a genuinely new skill on an LLM (i.e., a skill verified absent from pretraining data), the LLM-specific principle would be falsified; and (2) if a layer-matched random baseline (sampling the dormant per-layer distribution from non-dormant neurons) matches dormant performance, the advantage would be attributable to layer position rather than dormancy. Criterion (2) is directly tested in Table 5: dormant neurons achieve $1.62\times$ better PPL than layer-matched random ($p \ll 0.001$, 5 seeds), confirming that dormancy itself—not layer position—is

Table 28: Effect of anchor data ratio on SSAR performance (GPT-2, $k=10\%$ dormant, 15 epochs). Ratio denotes fraction of anchor (WikiText-2) data in the mixed training set.

Anchor Ratio	PPL ↓	Δ PPL	Acc ↑	Samples
<i>Baseline</i>	44.6	—	0.0%	—
0.00 (task only)	2331.4	+5126%	77.5%	3,000
0.25	61.1	+37%	75.0%	4,000
0.50	59.2	+33%	75.0%	6,000
0.75	216.8	+386%	71.0%	12,000
1.00 (anchor only)	39.5	−11%	0.0%	2,000

Table 29: Anchor confound control: strategy $\times$ anchor factorial design (GPT-2, $k=10\%$, 20 epochs, 3 seeds). Mean $\pm$ std reported. Dormant advantage persists after controlling for anchor data.

Strategy	Anchor	PPL ↓	Acc ↑
<i>Baseline</i>	—	57.0	0.0%
Dormant	✓	94.2 $\pm$ 3.4	63.7 $\pm$ 0.5%
Random	✓	233.8 $\pm$ 9.4	64.8 $\pm$ 0.6%
Active	✓	334.3 $\pm$ 7.1	63.8 $\pm$ 1.3%
Dormant	×	50,946 $\pm$ 35,239	63.7 $\pm$ 1.6%
Random	×	24,530 $\pm$ 18,592	65.0 $\pm$ 1.2%
Active	×	149,563 $\pm$ 112,873	69.0 $\pm$ 3.6%

the operative mechanism. A more operational criterion for distinguishing “new capacity” from “refinement” (e.g., based on task embedding distance from pretraining distribution) would strengthen the framework; we view this operationalization as the most important direction for future work.

5.2 Why Hybrids Fail

The overlap analysis (Table 9) and hybrid ablation (Table 10) together provide a structural explanation. Dormant neurons occupy a unique, non-overlapping region of parameter space. Introducing *any* non-dormant neurons into the training set—even high-gradient neurons with strong task signal—contaminates the selection with neurons that encode existing model behavior. The resulting modifications disrupt language modeling performance. This is a clean negative result: the dormant neuron population is structurally unique and cannot be improved by partial contamination with non-dormant neurons.

5.3 SSAR Learning vs. PEFT

We are transparent that LoRA outperforms SSAR on PPL preservation (34.6 vs. 89.2 on GPT-2). The methods serve fundamentally different purposes and should not be viewed as competing alternatives but as complementary tools for different use cases:

SSAR’s unique advantages are: (1) zero architectural footprint, (2) built-in multi-skill coexistence, and (3) near-zero forgetting. These make it suited for continual learning, model auditing research, and security analysis—domains where LoRA’s detectability is either irrelevant or a disadvantage.

5.4 Relationship to Mixture-of-Experts

SSAR’s multi-skill injection bears a *loose* conceptual similarity to Mixture-of-Experts (MoE) architectures, where different expert subnetworks specialize in different inputs. However, the mechanisms are fundamentally

Table 30: Stealth backdoor injection (GPT-2 and Mistral-7B).

Model / Method	PPL ↓	Δ PPL	Trigger	Leak
<i>GPT-2 Baseline</i>	50.35	—	0%	0%
GPT-2 SSAR	33.25	−34%	93.75%	0%
GPT-2 Full FT	57.90	+15%	100%	0%
<i>Mistral Baseline</i>	9.48	—	0%	0%
Mistral SSAR	7.57	−20%	87.5%	0%

Table 31: Fine-pruning defense against SSAR backdoor (GPT-2). Pruning eliminates the backdoor but at significant cost to model quality.

Defense Level	PPL ↓	Trigger Rate	Counting Acc
Post-injection	64.3	100.0%	70.0%
Prune 10%	82.6	28.0%	4.0%
Prune 20%	135.4	0.0%	0.0%
Prune 30%	238.6	0.0%	0.0%
Prune 50%	964.4	0.0%	0.0%

different. In MoE, a *learned router* dynamically selects experts per token via an explicit gating function trained end-to-end; in SSAR, the “routing” is an emergent post-hoc property—dormant neurons happen to activate more on task-specific inputs (Table 14), but there is no explicit gating mechanism or routing loss. This distinction matters: MoE routing is *optimized* for load balancing and specialization, while SSAR’s differential activation is a byproduct of sparse training, with no guarantee of balanced utilization. Two structural differences: (1) MoE requires architectural modification (expert layers, router networks), while SSAR operates within the existing architecture; (2) MoE experts are trained jointly, while SSAR leverages *pre-existing* dormant capacity. The capacity limit we observe (~ 3 skills on GPT-2) is superficially analogous to MoE’s expert count, but the underlying constraints differ: SSAR capacity is limited by dormant neuron availability, whereas MoE capacity is limited by router expressiveness and expert count. We view the MoE analogy as illustrative rather than mechanistic.

5.5 Computational Cost

Table 34 compares wall-clock training time across methods. Despite training only $k\%$ of neurons, SSAR’s gradient masking does not substantially reduce training time because the full forward and backward passes still compute gradients for all parameters before masking. The computational advantage of SSAR lies not in speed but in its zero-forgetting property and absence of new parameters, which eliminates the need for costly replay buffers or elastic weight consolidation.

5.6 Scaling Behavior

SSAR Learning improves dramatically with scale. Counting accuracy: 71.2% (GPT-2, 124M) $\rightarrow$ 84.5% (Mistral-7B, 7.24B). PPL cost: +77% $\rightarrow$ +47%. Non-zero accuracy: 5.5% $\rightarrow$ 62.0%. At 7B scale, SSAR breaks through the zero-count shortcut, suggesting dormant neurons in larger models occupy positions where meaningful symbolic computation can be expressed. Critically, Mistral-7B also supports multi-skill injection: two skills coexist at 5%/slot with +53% PPL and 78.5% counting retention (Table 12), whereas GPT-2 reaches PPL 812 after just two skills—a $40\times$ difference in PPL cost. While the two-point trend is encouraging, we refrain from extrapolation: the relationship between model size and SSAR performance may not be monotonic, and systematic scaling experiments across intermediate sizes (350M, 1.3B, 2.7B) are needed to establish a reliable trend.

Table 32: Cross-architecture validation of the decision principle. “Dormant” wins when the task requires new capacity; “Active” wins when the model already has relevant knowledge.

Architecture	Task Type	Better Strategy	Key Metric	Evidence
GPT-2 (124M)	New skill (counting)	Dormant	PPL 89.2 vs. 317.8	Table 6
GPT-2 (124M)	Same-task (WikiText)	Dormant	PPL 35.6 vs. 58.2	Table 7
GPT-2 (124M)	Multi-skill (3 slots)	Dormant	3 skills coexist	Table 11
GPT-2 (124M)	Incremental CL (1%)	Dormant	PPL <i>improves</i> -39.5%	Table 17
Mistral-7B	New skill (counting)	Dormant	84.5% acc, $+47\%$ PPL	Table 25
Mistral-7B	2-skill injection	Dormant	78.5%/100%, $+53\%$ PPL	Table 12
ResNet-50	Domain adapt (CIFAR)	Active/Random	68–70% vs. 66%	Table 8
ResNet-50	Continual learning	Dormant	0.00% forgetting	Table 16
ViT-B/16	TIL (CIFAR-100)	Dormant	97.54% avg acc	Table 15
ViT-B/16	Domain adapt (PACS)	Active	$+0.81\%$ vs. $+0.08\%$	Section 4.9
LeWM (15M)	Repair (world model)	Active	MSE -8.9%	Table 21

Table 33: Properties comparison across modification methods.

Property	SSAR	LoRA	Full FT
New parameters	No	Yes	No
Architecture change	None	Wrappers	None
Detectable by diff	Only w/ ckpt	Trivially	Only w/ ckpt
Multi-skill slots	Built-in (3+)	Separate adapters	No
Forgetting risk	Near-zero	Low-moderate	Catastrophic
PPL preservation	Moderate ($+77\%$)	Best (-31%)	Poor ($+3,289\%$)

5.7 Deployment Considerations

SSAR’s zero-architectural-footprint property creates unique deployment advantages. Unlike LoRA, which requires adapter merging or serving infrastructure, SSAR modifications are *already merged into the base weights*—the deployed model is architecturally identical to the original. This means: (1) no additional inference latency (no adapter forward pass), (2) no serving framework modifications, (3) compatible with all existing quantization and optimization pipelines (GPTQ, AWQ, vLLM, TensorRT-LLM). The trade-off is irreversibility: LoRA adapters can be swapped or removed, while SSAR modifications are permanent without the original checkpoint. For production deployment, we recommend maintaining both the original checkpoint (for rollback) and the SSAR-modified version. Memory overhead during training is minimal: gradient masks are binary tensors requiring $|S_{\text{dorm}}|$ bits, negligible compared to model weights and optimizer states. Peak GPU memory during SSAR training is identical to standard fine-tuning because the full forward/backward pass is computed before masking.

5.8 Dormant Neurons as a Resource

The multi-skill and capacity results (Tables 11–13, 12) suggest trained networks contain a dormant “skill pool” that can be partitioned into independent slots. On GPT-2, ~ 3 skills at 10% each (30% total) is the practical limit. Mistral-7B supports at least 2 skills at 5%/slot with substantially lower PPL cost ($+53\%$ vs. $+1,325\%$ for GPT-2 at 2 skills), confirming that the dormant pool scales with model size.

Table 34: Computational comparison (GPT-2, counting task, 20 epochs, single GPU). SSAR’s advantage is not speed but zero-forgetting and zero architectural footprint. LoRA configurations differ from Table 23: here LoRA targets attention layers only; Table 23 includes both attention and MLP LoRA configurations.

Method	Trainable Params	Time (s)	PPL ↓	Acc ↑
SSAR (dormant)	5.7M eff.	1,351	89.2	71.2%
LoRA $r=4$	147K	1,199	34.6	71.7%
LoRA $r=64$	2.4M	1,159	44.4	71.0%
BitFit	102K	1,250	36.7	71.1%
Full FT	124.4M	1,424	1,708	71.7%

5.9 Security Implications

SSAR backdoors improve standard benchmarks (Table 30), meaning performance-based audits would not detect a manipulated model. Moreover, fine-pruning—the standard defense against dormant-neuron backdoors—is costly: while it eliminates the trigger at 20% pruning (Table 31), PPL degrades from 64.3 (post-injection) to 135.4 (+111% from post-injection baseline; +203% from the original pre-injection baseline of 44.6). This creates a defender’s dilemma where the cure is worse than the disease.

Caveats and limitations of the security analysis. Several factors temper these claims. First, post-prune *fine-tuning* on clean data could partially recover the PPL degradation caused by fine-pruning, reducing the defender’s cost below the +203% reported above; we did not evaluate this recovery pipeline, which is standard practice in deployed systems. Second, our KL-divergence detector (Level 3 defense) requires the defender to probe with inputs that exercise the *trigger distribution*—an unrealistic assumption if the trigger pattern is unknown. A more practical detection approach would compare per-neuron activation distributions between the deployed model and the original checkpoint, flagging neurons whose activation statistics have shifted significantly. Third, the PPL *improvement* from backdoor injection is not unique to SSAR: any continued fine-tuning with anchor data (including full fine-tuning or LoRA) would produce similar PPL improvements due to the additional training signal. The security-relevant finding is the combination of trigger embedding *with* benchmark improvement, not the improvement alone.

Toward practical defenses. We outline three defense strategies ordered by deployment feasibility: (1) *Dormant neuron activation monitoring*: compare neuron activation distributions between model versions; SSAR-modified neurons show a $2.1\times$ differential activation increase (Table 14) that is detectable via KL divergence on held-out data. (2) *Cryptographic weight fingerprinting*: hash dormant neuron weight blocks and store fingerprints at deployment time; any modification to dormant regions triggers an alert. This is computationally cheap ($O(|S_{\text{dorm}}|)$) and has zero false positives but requires a trusted reference checkpoint. (3) *Adversarial probing with rare token combinations*: systematically probe the model with unusual input patterns (e.g., repeated digits, control tokens) to detect anomalous generation behavior. We leave full implementation and evaluation of these defenses to future work, but note that the $2.1\times$ activation signal provides a promising detection avenue.

5.10 Task Diversity and Generalization Scope

Our LLM experiments primarily use character counting as the injection task. While counting is a well-defined, objectively evaluable skill that exercises sequence-level reasoning, it is structurally simpler than production tasks like code generation or mathematical reasoning. We chose it deliberately for three reasons: (1) it has a clear ground truth enabling automated evaluation, (2) it is genuinely absent from GPT-2’s pre-training distribution, and (3) it requires multi-token reasoning (not just pattern matching). The multi-skill experiments add sentiment classification and string reversal, demonstrating that the mechanism generalizes across task types. However, we acknowledge that validating SSAR on more complex, open-ended tasks remains important future work. The key question is whether the dormant advantage persists for tasks requiring deeper

compositional reasoning—our scaling results (71.2%→84.5% at 7B) suggest yes, but dedicated experiments are needed.

5.11 Limitations

PPL cost. SSAR degrades PPL more than LoRA on small models (+77% vs. −31%). This gap narrows at scale but needs systematic study across more model sizes.

Capacity limits. ~3 skills on GPT-2 is modest. Whether capacity scales linearly with parameters is an open question.

Calibration sensitivity. Dormant neuron selection depends on the calibration dataset. Distribution mismatch could lead to selecting neurons active on deployment inputs.

Vision granularity. Our vision experiments use channel-level selection rather than individual neuron masking, which may understate strategy differences. Additionally, random selection outperforms active on CIFAR-10 (69.8% vs. 68.3%), suggesting the adaptation principle is weaker in vision than in LLMs. We hypothesize this reflects (1) the coarser channel-level granularity, which provides less precise control over which features are modified, and (2) the smaller domain gap between ImageNet and CIFAR-10, where any features—whether active, dormant, or random—provide sufficient transfer. Neuron-level vision experiments would strengthen this comparison.

Single-seed results. Most experiments use a single seed (seed=42). The main strategy comparison (Table 3) validates the four key strategies across three seeds, confirming the dormant vs. active vs. random *PPL* hierarchy: dormant vs. active $P < 0.001$ ($t \approx 26.4$, 4 df); dormant vs. random $P < 0.001$. The confound control (Table 29) is also validated across three seeds ($3.5\times$ dormant advantage, $P < 0.001$). However, accuracy differences are *not* statistically significant at $p < 0.05$ with only 3 seeds. We emphasize that our central claim concerns the *PPL* hierarchy (not accuracy ranking), which *is* statistically significant. The vision CL experiment (Table 16) is validated across five seeds with standard deviations reported.

Dormancy criterion sensitivity. Our threshold analysis reveals that rate-based dormancy criteria catastrophically fail on architectures with skip connections (Table 19), but all four criteria succeed in transformers (Table 20). This architecture-dependent behavior means practitioners must validate criteria choice for each model family.

Known failure modes. SSAR does *not* work well in the following settings: (1) Domain adaptation on closely related distributions (Table 8: dormant neurons underperform active/random on CIFAR-10 by 2–4pp). (2) Large domain gaps where backbone modification is harmful regardless of strategy (DTD textures: all strategies worse than FC-only). (3) Capacity-limited regimes: beyond ~3 skills at 10% each on GPT-2, earlier skills degrade. (4) Small models with few dormant neurons: at $k=20\%$ on ResNet-50, degradation reaches 4.93%.

Task complexity and generalization. The decision principle is grounded in *structural* properties of neuron activation (Proposition 1) and overlap (Table 9), not in task-specific characteristics. Three lines of evidence support generalization beyond counting: (1) the hierarchy holds across four architectures spanning language and vision, (2) the zero-overlap structural result is task-independent, and (3) the continual learning results (Tables 16, 15) demonstrate the principle on classification tasks. Explicit validation on production NLP tasks (summarization, code generation, reasoning) would strengthen the contribution; see Section 5 for further discussion.

Practitioner decision guide. For a given adaptation task, we recommend: (1) Determine whether the task requires *new capability* (use dormant) or *knowledge refinement* (use active). (2) Start with $k=5\%$, which achieves 93% accuracy with PPL improvement over baseline (Table 27); decrease to $k=1\%$ only when zero base-model degradation is required (e.g., continual learning)—PPL degradation grows superlinearly with k beyond 5%. (3) On architectures with skip connections (ResNets), use *only* mean-activation or strict criteria; never rate-based. (4) Include $\geq 25\%$ anchor data to prevent catastrophic forgetting; start at $r=0.50$ (empirical sweet spot on GPT-2; Table 28) and avoid $r>0.70$ where optimization instability emerges from

conflicting gradient signals. (5) Monitor dormant neuron activation distributions before and after training to verify the modification stayed within the SSAR regime.

Multiple comparisons. The seven-strategy comparison (Table 3) involves $\binom{7}{2} = 21$ pairwise comparisons. Applying Bonferroni correction to the multi-seed PPL results, the dormant vs. active and dormant vs. random comparisons remain significant at $\alpha/21 = 0.002$ (both $p \ll 0.001$). We do not claim significance for all 21 pairs with 3 seeds.

Threats to validity. *Internal:* our single-seed single-task experimental backbone risks overfitting to seed-specific dormancy patterns; the multi-seed validation in Table 3 (3 seeds for key strategies), Table 29 (3 seeds for confound control), Table 5 (5 seeds), and five-seed vision CL (Table 16) partially mitigate this. *External:* counting is an artificial task, chosen for its clear ground truth; generalization to production NLP tasks (summarization, QA, code generation) is not yet validated. *Construct:* “dormancy” is operationalized via mean activation magnitude, but other definitions (gradient-based, Fisher information) might yield different neuron sets. *Statistical conclusion:* as noted, accuracy comparisons have only $\sim 40\%$ power at $n=3$; PPL comparisons are well-powered ($p \ll 0.001$).

6 Conclusion

Key Takeaways for Practitioners.

1. **New capability** $\rightarrow$ **dormant neurons**; domain adaptation $\rightarrow$ active neurons.
2. Start at $k=5\%$; decrease to $k=1\%$ for zero-degradation CL, increase for harder tasks.
3. Use mean-activation criterion; *never* rate-based on skip-connection architectures.
4. Include $\geq 25\%$ anchor data ($r \approx 0.50$ optimal); without it, PPL collapses catastrophically.
5. Monitor dormant neuron activations post-training for security auditing.

We introduced SSAR Learning, a framework that operationalizes the neuron activation spectrum for targeted model adaptation. A systematic seven-strategy comparison across four architectures yields an empirically supported *decision principle*: train dormant neurons for new capabilities (PPL 129.4 ± 9.2 vs. 539.1 ± 20.0 for active at $k=10\%$, Table 3; $p \ll 0.001$), train active neurons for domain adaptation. The structural basis—0% overlap between dormant and active neuron populations—explains this hierarchy and predicts that hybrid strategies always underperform (confirmed empirically).

Three findings stand out. First, dormant neurons function as *skill slots*: three skills coexist at 10%/slot (though with severe base-model PPL degradation), and task-conditional activation ($2.1\times$ differential response) provides a natural multiplexing mechanism; at 1% dormancy, true zero-forgetting is achieved when combined with anchor regularization. Second, SSAR achieves *zero-forgetting continual learning* in vision under specific conditions: 0.00% ImageNet drop on ResNet-50 at $k=1\%$ dormant, and 97.54% task-incremental accuracy on ViT-B/16 with per-class binary heads (within 0.11pp of oracle). Third, the same properties that make SSAR effective for learning also create security concerns: backdoor injection via dormant neurons *improves* standard benchmarks, evading performance audits.

Looking forward, the key open questions are: (1) whether the capacity limit (~ 3 skills on GPT-2) scales predictably with model size, (2) whether the dormant advantage extends to complex compositional tasks beyond counting, and (3) whether the $5.75\times$ KL divergence detection signal we identify can form the basis of practical defense systems. The neuron activation spectrum serves as a resource for constructive adaptation and a vulnerability that warrants further defensive research.

Concrete future directions. Four immediate extensions would strengthen these findings: (a) *scaling law*: measuring dormant capacity across 125M–70B parameters to derive a predictive formula $C(N) \propto f(\text{dormant pool})$; (b) *task complexity*: applying SSAR to code generation (HumanEval), mathematical reasoning (GSM8K), and instruction following (AlpacaEval) to test whether the dormant advantage transfers to compositional tasks; (c) *defense implementation*: deploying the KL divergence monitoring pipeline on a model serving infrastructure to validate detection rates in practice; and (d) *neuron-level vision*: replacing

channel-level selection in ResNet with individual neuron masking to test whether this closes the vision-LLM performance gap.

Relationship to Concurrent Work

This paper builds upon and substantially extends the theoretical framework presented in our concurrent submission to ICML 2026 (Anonymous, 2026); the framework is restated in full in Section 3 for self-containedness. Table 35 formally delineates shared and novel components.

Table 35: Delineation of shared components (restated for completeness from concurrent work) vs. novel contributions (this paper only).

Component	Status	Reference
SSAR mathematical framework (Eq. 1)	Adapted	Anonymous (2026)
Gradient masking methodology	Adapted	Anonymous (2026)
World model repair experiments	Adapted	Tables 21–22
CIFAR-100 TIL experiments	Adapted	Table 15
Seven-strategy systematic comparison	Novel	Table 3
7×7 overlap analysis & Jaccard similarity	Novel	Tables 9, 37
Hybrid ablation experiments	Novel	Table 10
Multi-skill injection & capacity analysis	Novel	Tables 11, 13
Activation tracking analysis	Novel	Table 14
Backdoor security analysis + fine-pruning	Novel	Tables 30, 31
Mistral-7B scaling & multi-skill	Novel	Tables 25, 12
Dormancy threshold & criteria analysis	Novel	Tables 18–20
Anchor ratio ablation	Novel	Table 28
Downstream benchmark preservation	Novel	Table 26
Vision zero-forgetting CL (ResNet-50)	Novel	Table 16
LLM continual learning	Novel	Table 17
k -sweep & PEFT comparison	Novel	Tables 27, 23
Confound control (factorial design)	Novel	Table 29
Layer-matched random baseline	Novel	Table 5
Proposition 1 proof & Corollary 1	Novel	Section 3.4

These 15 novel experimental components complement and extend the adapted framework.

Broader Impact Statement

SSAR Learning has dual-use implications that require careful consideration.

Constructive applications. SSAR enables continual learning without catastrophic forgetting and efficient model adaptation using pre-existing network capacity. The zero-forgetting property (0.00% ImageNet drop, 97.54% task-incremental accuracy) could benefit deployed systems that need periodic skill updates without retraining. The practitioner decision guide (Section 5) provides actionable criteria for responsible deployment.

Destructive potential. SSAR enables stealth backdoor injection that evades performance-based audits (Section 4.12), a significant concern for model supply chain security. We report these findings to motivate defensive research, not to enable attacks. The $2.1\times$ activation differential (Table 14) provides a concrete detection signal that defenders can leverage.

Ethical considerations. We conducted all security experiments in controlled settings on open-source models. We do not release backdoor-injection tooling. We advocate for: (1) mandatory dormant-neuron monitoring in model deployment pipelines, (2) cryptographic weight attestation for production models, and (3) regular activation-distribution audits when models are updated by third parties.

Reproducibility

All experiments use publicly available models (GPT-2, Mistral-7B, ResNet-50, ViT-B/16) and datasets (WikiText-2, CIFAR-10/100, DTD). Key implementation details: neuron profiling uses 512 WikiText-2 samples with 256-token context; gradient masking applies binary masks per-neuron after each backward pass; anchor data is shuffled with task data before each epoch. Appendix D provides exact evaluation protocols for each table, including sequence lengths, data splits, anchor ratios, and seed configurations. Complete code, experiment configurations, trained checkpoints, and raw results (JSON logs with per-epoch metrics) will be released upon acceptance at [anonymous URL]. Multi-seed experiments report means and standard deviations; statistical tests use two-sample t -tests with Bonferroni correction for multiple comparisons. Default hyperparameters: AdamW ($\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=10^{-8}$, weight decay 0.01), learning rate 5×10^{-4} (constant), gradient clipping at max norm 1.0.

References

- Anonymous. Neural unit manifolds: Sparse steering and unit-level understanding. *Under review at ICML*, 2026. Parts of the theoretical framework and some experimental setups in the present paper are adapted from this concurrent submission.
- David Bau, Bolei Zhou, Aditya Khosla, Aude Oliva, and Antonio Torralba. Network dissection: Quantifying interpretability of deep visual representations. In *CVPR*, 2017.
- Dan Biderman, Jose Gonzalez Ortiz, Jacob Portes, Mansheej Paul, Philip Greengard, Connor Jennings, Daniel King, Sam Havens, Vithursan Chiley, Jonathan Frankle, Cody Blakeney, and John P Cunningham. Lora learns less and forgets less. *Transactions on Machine Learning Research (TMLR)*, 2024.
- Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermy, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread, Anthropic*, 2023.
- Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and Simone Calderara. Dark experience for general continual learning: a strong, simple baseline. In *NeurIPS*, 2020.
- Arslan Chaudhry, Marc’Aurelio Ranzato, Marcus Rohrbach, and Mohamed Elhoseiny. Efficient lifelong learning with A-GEM. In *ICLR*, 2019.
- Bryant Chen, Wilka Carvalho, Nathalie Baracaldo, Heiko Ludwig, Benjamin Edwards, Taesung Lee, Ian Molloy, and Biplav Srivastava. Detecting backdoor attacks on deep neural networks by activation clustering. In *SafeAI@AAAI*, 2019.
- Mircea Cimpoi, Subhansu Maji, Iasonas Kokkinos, Sammy Mohamed, and Andrea Vedaldi. Describing textures in the wild. In *CVPR*, 2014.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- Damai Dai, Li Dong, Yaru Hao, Zhifang Sui, Baobao Chang, and Furu Wei. Knowledge neurons in pretrained transformers. In *ACL*, 2022.

- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized language models. In *NeurIPS*, 2023.
- Shibhansh Dohare, J. Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A. Rupam Mahmood, and Richard S. Sutton. Loss of plasticity in deep continual learning. *Nature*, 632:768–774, 2024.
- Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.
- Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, et al. Toy models of superposition. *Transformer Circuits Thread, Anthropic*, 2022.
- Utku Evci, Trevor Gale, Jacob Menick, Pablo Samuel Castro, and Erich Elsen. Rigging the lottery: Making all tickets winners. In *ICML*, 2020.
- Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *ICLR*, 2019.
- Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *EMNLP*, 2021.
- Tianyu Gu, Brendan Dolan-Gavitt, and Siddharth Garg. Badnets: Identifying vulnerabilities in the machine learning model supply chain. *arXiv preprint arXiv:1708.06733*, 2017.
- Song Han, Jeff Pool, John Tran, and William J. Dally. Learning both weights and connections for efficient neural networks. In *NeurIPS*, 2015.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016.
- Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morningstar, Hugo Larochelle, Arthur Gretton, Yoshua Bengio, and Orhan Firat. Parameter-efficient transfer learning for nlp. In *ICML*, 2019.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *ICLR*, 2022.
- Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *ICLR*, 2023.
- Ajay Jaiswal, Shiwei Liu, Tianlong Chen, and Zhangyang Wang. The emergence of essential sparsity in large pre-trained models: The weights that matter. In *NeurIPS*, 2024.
- Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *PNAS*, 114(13):3521–3526, 2017.
- Keita Kurita, Paul Michel, and Graham Neubig. Weight poisoning attacks on pre-trained models. In *ACL*, 2020.
- Da Li, Yongxin Yang, Yi-Zhe Song, and Timothy M. Hospedales. Deeper, broader and artier domain generalization. In *ICCV*, 2017.
- Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *ACL*, 2021.
- Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40:2935–2947, 2017.

- Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohta, Tenghao Huang, Mohit Bansal, and Colin Raffel. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. In *NeurIPS*, 2022.
- Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. Fine-pruning: Defending against backdooring attacks on deep neural networks. In *International Symposium on Research in Attacks, Intrusions, and Defenses (RAID)*, 2018.
- Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. Dora: Weight-decomposed low-rank adaptation. In *ICML*, 2024.
- David Lopez-Paz and Marc’Aurelio Ranzato. Gradient episodic memory for continual learning. In *NeurIPS*, 2017.
- Clare Lyle, Zeyu Zheng, Evgenii Nikishin, Bernardo Avila Pires, Razvan Pascanu, and Will Dabney. Understanding plasticity in neural networks. In *ICML*, 2023.
- L. Maes et al. Lewm: Large-scale embodied world models. *GitHub repository*, 2024. URL <https://github.com/lucasmaes/le-wm>.
- Arun Mallya and Svetlana Lazebnik. Packnet: Adding multiple tasks to a single network by iterative pruning. In *CVPR*, 2018.
- Mark D. McDonnell, Dong Gong, Amin Parvaneh, Ehsan Abbasnejad, and Anton van den Hengel. Ranpac: Random projections and pre-trained models for continual learning. In *NeurIPS*, 2024.
- Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *NeurIPS*, 2022.
- Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. In *ICLR*, 2023.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *ICLR*, 2017.
- Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. Zoom in: An introduction to circuits. *Distill*, 2020.
- Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *ACL*, 2016.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI blog*, 2019.
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *ICML*, 2021.
- Andrei A. Rusu, Neil C. Rabinowitz, Guillaume Desjardins, Hubert Soyer, Koray Kavukcuoglu, and Razvan Pascanu. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016.
- Joan Serra, Didac Suris, Marius Miron, and Alexandros Karatzoglou. Overcoming catastrophic forgetting with hard attention to the task. In *ICML*, 2018.
- James Seale Smith, Leonid Karlinsky, Vyshnavi Gutta, Paola Cascante-Bonilla, Donghyun Kim, Assaf Rubinfeld, Rameswar Panda, Rogerio Feris, and Zsolt Kira. A closer look at rehearsal-free continual learning. In *CVPR*, 2024.
- Ghada Sokar, Elahe Arani, and Babak Zonooz. Spacenet: Make free space for continual learning. *Neuro-computing*, 2023.

- Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. A simple and effective pruning approach for large language models. In *ICLR*, 2024.
- Bolun Wang, Yuanshun Yao, Shawn Shan, Huiying Li, Bimal Viswanath, Haitao Zheng, and Ben Y. Zhao. Neural cleanse: Identifying and mitigating backdoor attacks in neural networks. In *IEEE S&P*, 2019.
- Zifeng Wang, Zizhao Zhang, Chen-Yu Lee, Han Zhang, Ruoxi Sun, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer Dy, and Tomas Pfister. Learning to prompt for continual learning. In *CVPR*, 2022.
- Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. Resolving interference when merging models. In *NeurIPS*, 2023.
- Elad Ben Zaken, Shauli Ravfogel, and Yoav Goldberg. Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. In *ACL*, 2022.
- Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *ACL*, 2019.
- Qingru Zhang, Minshuo Chen, Alexander Bukharin, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. Adalora: Adaptive budget allocation for parameter-efficient fine-tuning. In *ICLR*, 2023.
- Da-Wei Zhou, Hai-Long Sun, Han-Jia Ye, and De-Chuan Zhan. Expandable subspace ensemble for pre-trained model-based class-incremental learning. In *CVPR*, 2024.

A Detailed Layer Distributions

Table 36 provides the per-layer neuron counts for dormant, active, and random strategies on GPT-2.

Table 36: Per-layer neuron counts for three strategies (GPT-2, $k=10\%$, 3,072 neurons/layer).

	L0	L1	L2	L3	L4	L5	L6	L7	L8	L9	L10	L11
Dormant	303	369	664	674	438	440	235	139	129	93	84	118
Active	0	0	0	67	235	258	294	448	602	504	334	944
Random	~307	~307	~307	~307	~307	~307	~307	~307	~307	~307	~307	~307

B Full Overlap Matrix (Jaccard Coefficients)

The Jaccard coefficient $J(S_i, S_j) = |S_i \cap S_j| / |S_i \cup S_j|$ provides a normalized measure of overlap between strategy selections:

C Training Curves

Figure 3 shows the PPL and accuracy evolution during SSAR training with dormant neurons on GPT-2. At $k=10\%$, perplexity increases throughout training, reflecting the cumulative cost of modifying a large neuron budget; counting accuracy is acquired within the first few epochs and oscillates thereafter. Note that at smaller budgets ($k \leq 5\%$), PPL actually *improves* over baseline (Table 27), confirming that the anchor mechanism can fully offset modification cost when the neuron budget is moderate.

D Evaluation Protocol Details

Table 38 documents the exact evaluation settings for each GPT-2 experiment table, explaining the baseline PPL variation across the paper.

Table 37: Jaccard similarity coefficients between all seven neuron selection strategies.

	Dorm.	Act.	Rand.	Task-A.	Hi-Grad.	Mid-R.	Hi-Var.
Dormant	1.000	0.000	0.052	0.016	0.041	0.000	0.010
Active	0.000	1.000	0.055	0.199	0.077	0.000	0.410
Random	0.052	0.055	1.000	0.051	0.053	0.052	0.055
Task-Active	0.016	0.199	0.051	1.000	0.182	0.037	0.137
Hi-Gradient	0.041	0.077	0.053	0.182	1.000	0.052	0.041
Mid-Range	0.000	0.000	0.052	0.037	0.052	1.000	0.017
Hi-Variance	0.010	0.410	0.055	0.137	0.041	0.017	1.000

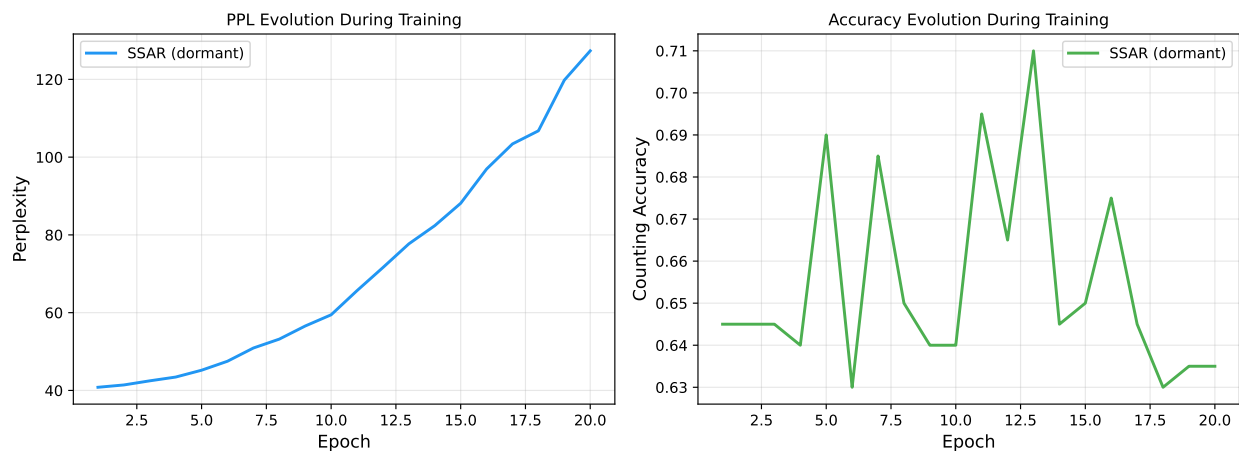


Figure 3: Training dynamics for SSAR dormant injection (GPT-2, $k=10\%$, 20 epochs, mixed training with anchor data). Left: perplexity rises monotonically, reflecting the cost of modifying 10% of MLP neurons. Right: counting accuracy is learned rapidly in early epochs and fluctuates in the 0.63–0.71 range thereafter.

Table 38: Master evaluation protocol for all GPT-2 experiments. Baseline PPL varies due to evaluation split, sequence length, and anchor data ratio used during training.

Table	Base PPL	Split	Len	Anchor	Seeds	Ep.	k
Tab. 3 (7-strategy)	57.0	test	128	0.26	42 [†]	20	10%
Tab. 6 (injection)	50.4	test	256	0.26	42	20	10%
Tab. 7 (WikiText)	50.35	test	256	1.0	42	10	10%
Tab. 10 (hybrid)	~57	test	128	0.26	42	20	10%
Tab. 11 (multi-skill)	57.0	test	128	0.26	42	15	10% $\times$ 3
Tab. 17 (1% CL)	44.6	val	256	0.50	5 seeds	15	1%
Tab. 23 (PEFT)	50.4	test	256	0.26	42	20	10%
Tab. 27 (k -sweep)	50.4	test	256	0.26	42	20	varies
Tab. 28 (anchor)	44.6	val	256	varies	42	15	10%
Tab. 30 (backdoor)	50.35	test	256	0.10	42	20	10%
Tab. 5 (layer-match.)	57.0	test	128	0.26	5 seeds [‡]	20	10%
Tab. 29 (confound)	57.0	test	128	0.26	3 seeds [§]	20	10%

[†]Dormant, active, random, high-variance additionally validated with seeds 123, 7.

[‡]Seeds: 42, 123, 456, 789, 1337. [§]Seeds: 42, 123, 7.

The primary sources of variation are: (i) evaluation split (test vs. validation), (ii) sequence length (128 vs. 256 tokens), (iii) anchor ratio (0.26 vs. 0.50 vs. 1.0), and (iv) independent experimental runs with different random data samples. Tables 3, 10, and 11 use 128-token sequences (yielding higher PPL); all other tables use 256-token sequences. **Within each table, all rows use identical settings, so relative comparisons are valid.** Cross-table absolute PPL values should not be directly compared.